

Deriving the Gradients of Some Popular Optimal Transport Algorithms

Fangzhou Xie

Department of Economics, Rutgers University

April 14, 2025

Abstract

In this note, I review entropy-regularized Monge-Kantorovich problem in Optimal Transport, and derive the gradients of several popular algorithms popular in Computational Optimal Transport, including the Sinkhorn algorithms, Wasserstein Barycenter algorithms, and the Wasserstein Dictionary Learning algorithms.

Contents

1	Introduction	4
2	Monge-Kantorovich Problem and Its Entropic Regularization	5
3	Sinkhorn Algorithms	7
3.1	Vanilla Sinkhorn Algorithm	7
3.2	Parallel Sinkhorn Algorithm	8
3.3	Log-stabilized Sinkhorn	10
4	Sinkhorn Algorithms with Gradients	13
4.1	Vanilla Sinkhorn	13
4.2	Parallel Sinkhorn	16
4.3	Log-Stabilized Sinkhorn	18
5	Wasserstein Barycenter	20
5.1	Parallel Algorithm	21
5.2	Log-stabilized Algorithm	22
6	Wasserstein Barycenter with Jacobians	25
6.1	Jacobians for Parallel Algorithm	25
6.2	Jacobians for Log-Stabilized Algorithm	27
7	Wasserstein Dictionary Learning	30
7.1	Numeric Optimization Methods	30
7.2	Wasserstein Dictionary Learning	32
7.3	WDL Algorithm	35
	References	37
	Appendices	40
	Appendix A Mathematical Notations	40
	Appendix B Lemmas	41

List of Algorithms

3.1	Vanilla Sinkhorn Algorithm	8
3.2	Parallel Sinkhorn Algorithm	9
3.3	Log-Stabilized Sinkhorn Algorithm	12
4.1	Vanilla Sinkhorn Algorithm with Gradient	16
4.2	Parallel Sinkhorn Algorithm with Jacobians	18
4.3	Log-Stabilized Sinkhorn with Gradient	20
5.1	Parallel Wasserstein Barycenter Algorithm	22
5.2	Log-Stabilized Wasserstein Barycenter Algorithm	25
6.1	Parallel Wasserstein Barycenter Algorithm with Jacobians	27
6.2	Log-Stabilized Wasserstein Barycenter Algorithm with Jacobians	30
7.1	Wasserstein Dictionary Learning Algorithm	36

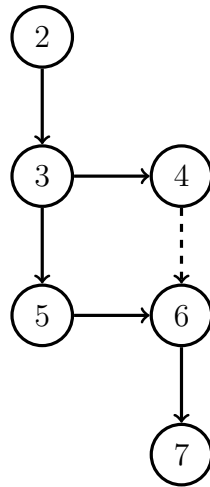


Figure 1: Diagram of the structure of the paper.

1 Introduction

The French mathematician Gaspard Monge (1746 – 1818) was allegedly interested in the question of how to efficiently dig dirt from the earth and use it to patch the castle wall, i.e. moving dirt from one location to another¹. Therefore, the question of how to “transporting” dirt efficiently became the start of the study of Optimal Transport. The field has received much attention over the years, because it is deeply connected to many interesting topics across different fields, including but not limited to Linear Programming, PDE, statistics, image processing, natural language processing, etc. Interested readers should refer to the now standard monographs by Villani (2003; 2008) for the mathematical theories, Santambrogio (2015) for applied mathematicians, Galichon (2016) for economists, and Peyré and Cuturi (2019) for computational aspects of OT.

This note can be seen as a quick review of some basic algorithms on Computational Optimal Transport, and provide the manually-derived gradients and Jacobians for the said algorithms. This will also form the foundation for the companion R package² *wig*, which provides an efficient implementation of the computation (OT algorithms and their gradients/Jacobians) laid out in this note and also the WIG models (Xie, 2020a; Xie, 2020b).

Figure 1 provides a diagram of the structure of the paper. For readers interested in entropic regularized OT problems, they can refer to Sections 2 and 3 with optional Section 4; for readers interested in Wasserstein Barycenter problems, they can refer to Sections 2, 3 and 5 with optional Section 6; for readers interested in Wasserstein Dictionary Learning, they can refer to Sections 2, 3 and 5 to 7.

All the gradients and Jacobians in this note are derived following the Magnus–Neudecker notation on matrix calculus (Magnus and Neudecker, 2019). The algorithms are also checked numerically by the Julia Zygote³ and ForwardDiff⁴ library to ensure the correct derivation.

The remainder of this paper will be organized as follows. Section 2 briefly reviews the Monge–Kantorovich problem, its entropic regularized problem and solution. Section 3 introduces the Sinkhorn algorithm and its variants to solve the entropic regularized OT problem. Section 4 derives the gradients and Jacobians of the family of Sinkhorn algorithms. Section 5 discusses the Wasserstein Barycenter problem. Section 6 shows the gradient of Sinkhorn-like algorithms to solve the Wasserstein Barycenter problem. Section 7 considers the Wasserstein Dictionary Learning problem. Appendix A and Appendix B lists all the mathematical notations and lemmas to simplify the expressions used throughout this note.

¹“Lorsqu’on doit tranfporter des terres d’un lieu dans un autre, on a contume de donner le nom de *Déblai* au volume des terres que l’on doit tranfporter, & le nom de *Remblai* à l’dfpace qu’elles doivent occuper après le tranfport”. – Gaspard Monge (1781). “Memoire Sur La Theorie Des Deblais et Des Remblais.” In: *Histoire de l’Academie Royale des Sciences de Paris*.

²<https://github.com/fangzhou-xie/wig>

³<https://github.com/FluxML/Zygote.jl> (Innes, 2019)

⁴<https://github.com/JuliaDiff/ForwardDiff.jl> (Revels, Lubin, and Papamarkou, 2016)

2 Monge-Kantorovich Problem and Its Entropic Regularization

The Monge–Kantorovich problem⁵ states that: given two probability vectors⁶ $\mathbf{a} \in \Sigma_M$ and $\mathbf{b} \in \Sigma_N$, how to find a *coupling matrix* $\mathbf{P} \in \mathbb{R}_+^{M \times N}$ where each $\mathbf{P}_{ij}$ describes the flow of masses from bin i to bin j , such that the total cost of moving masses are optimal (minimal). The cost of moving one unit of mass from bin i to bin j is $\mathbf{C}_{ij}$ and $\mathbf{C} \in \mathbb{R}_+^{M \times N}$.

Problem 1 (Monge–Kantorovich). *Let $\mathbf{a} \in \Sigma_M$ and $\mathbf{b} \in \Sigma_N$. The set of all coupling matrices is:*

$$\mathbf{U}(\mathbf{a}, \mathbf{b}) \equiv \{\mathbf{P} \in \mathbb{R}_+^{M \times N} : \mathbf{P} \cdot \mathbf{1}_N = \mathbf{a} \text{ and } \mathbf{P}^\top \cdot \mathbf{1}_M = \mathbf{b}\}. \quad (1)$$

Given a cost matrix $\mathbf{C} \in \mathbb{R}^{M \times N}$, the Kantorovich optimal transport problem tries to find the solution of the following

$$\ell_{\mathbf{C}}(\mathbf{a}, \mathbf{b}) \equiv \min_{\mathbf{P} \in \mathbf{U}(\mathbf{a}, \mathbf{b})} \langle \mathbf{C}, \mathbf{P} \rangle = \sum_{i,j} \mathbf{C}_{ij} \mathbf{P}_{ij}. \quad (2)$$

To solve this problem in practice, one needs to resort to linear programming⁷, which can be very challenging when the problem becomes large enough. Instead of finding the exact solution to the above problem, one can actually add an entropy regularization term to make this problem convex and hence greatly ease the computation. This was first proposed in the seminal work of Cuturi (2013) which leverages Sinkhorn–Knopp double scaling algorithm⁸(Sinkhorn, 1964; Sinkhorn and Knopp, 1967; Knight, 2008).

Problem 2 (Entropic Regularized OT Problem, Cuturi (2013) and Peyré and Cuturi (2019)). *Given the coupling matrix $\mathbf{P} \in \mathbb{R}_+^{M \times N}$, its discrete entropy is defined as*

$$\mathbf{H}(\mathbf{P}) \equiv -\langle \mathbf{P}, \log(\mathbf{P}) - \mathbf{1} \rangle = -\sum_{i,j} \mathbf{P}_{ij}(\log(\mathbf{P}_{ij}) - 1).$$

⁵This is also called Kantorovich relaxation of the original Monge problem, as the problem was first proposed by Monge (1781) and then relaxed by Kantorovich (1942). For mathematical foundation of Optimal Transport theory, the classic references are Villani (2003) and Villani (2008); for its application in Economics, see Galichon (2016); for an introduction for applied mathematicians, see Santambrogio (2015); for the computational aspects (algorithms and their properties) and its applications in data science and machine learning, see Peyré and Cuturi (2019).

⁶Sometimes also called “histograms” in the Computational Optimal Transport community, for example in Peyré and Cuturi (2019).

⁷See Peyré and Cuturi (2019, Chapter 3) for a historical overview and related algorithms.

⁸Therefore, in Optimal Transport literature, people usually refer to the numeric algorithm to solve the entropic regularized Kantorovich problem as the “Sinkhorn algorithm.” Hence, its extensions are also named as “X-khorn algorithms”, for example “Greenkhorn algorithm” (Altschuler, Weed, and Rigollet, 2017) or “Screenkhorn” (Alaya, Bézar, Gasso, and Rakotomamonjy, 2019). Because of its simplicity, it has been re-discovered multiple times in history and renamed accordingly, for example, it can be linked to Schrödinger bridge problem (Schrödinger, 1931) or the RAS method (Bacharach, 1970). See Knight (2008), Léonard (2013), and Modin (2024) for historical accounts.

With the cost matrix $\mathbf{C}$, the entropic regularized loss function is

$$\ell_{\mathbf{C}}^{\varepsilon}(\mathbf{a}, \mathbf{b}) \equiv \langle \mathbf{P}, \mathbf{C} \rangle - \varepsilon \mathbf{H}(\mathbf{P}), \quad (3)$$

and the entropic regularized OT problem is thus

$$\min_{\mathbf{P} \in \mathbf{U}(\mathbf{a}, \mathbf{b})} \ell_{\mathbf{C}}^{\varepsilon}(\mathbf{a}, \mathbf{b}) \equiv \min_{\mathbf{P} \in \mathbf{U}(\mathbf{a}, \mathbf{b})} \langle \mathbf{P}, \mathbf{C} \rangle - \varepsilon \mathbf{H}(\mathbf{P}). \quad (4)$$

The Sinkhorn problem has a unique optimal solution, since it is ε -strongly convex (Peyré and Cuturi, 2019).

Proposition 3 (Uniqueness of the Entropic OT Solution). *The solution to Equation (4) is unique and has the form*

$$\mathbf{P}_{ij} = \mathbf{u}_i \mathbf{K}_{ij} \mathbf{v}_j, \quad (5)$$

or in matrix notation

$$\mathbf{P} = \text{diag } \mathbf{u} \cdot \mathbf{K} \cdot \text{diag } \mathbf{v} \quad (6)$$

where $i \in \{1, \dots, M\}$ and $j \in \{1, \dots, N\}$ with two (unknown) scaling variables $\mathbf{u} \in \mathbb{R}^M$ and $\mathbf{v} \in \mathbb{R}^N$.

Proof. This is essentially Proposition 4.3 in Peyré and Cuturi (2019). Let $\mathbf{f} \in \mathbb{R}^M$ and $\mathbf{g} \in \mathbb{R}^N$ be two dual variables for the two constraints $\mathbf{P} \cdot \mathbf{1}_N = \mathbf{a}$ and $\mathbf{P}^\top \cdot \mathbf{1}_M = \mathbf{b}$, respectively. The Lagrangian of Equation (4) becomes

$$\mathcal{L}(\mathbf{P}, \mathbf{f}, \mathbf{g}) = \langle \mathbf{P}, \mathbf{C} \rangle - \varepsilon \mathbf{H}(\mathbf{P}) - \langle \mathbf{f}, \mathbf{P} \cdot \mathbf{1}_m - \mathbf{a} \rangle - \langle \mathbf{g}, \mathbf{P}^\top \cdot \mathbf{1}_n - \mathbf{b} \rangle. \quad (7)$$

The FOC (first order condition) gives us

$$\frac{\partial \mathcal{L}(\mathbf{P}, \mathbf{f}, \mathbf{g})}{\partial \mathbf{P}_{ij}} = \mathbf{C}_{ij} + \varepsilon \log(\mathbf{P}_{ij}) - \mathbf{f}_i - \mathbf{g}_j = 0$$

Therefore we can solve for the optimal coupling matrix $\mathbf{P}$ as:

$$\log(\mathbf{P}_{ij}) = \frac{\mathbf{f}_i}{\varepsilon} \left(-\frac{\mathbf{C}_{ij}}{\varepsilon} \right) \frac{\mathbf{g}_j}{\varepsilon}$$

or

$$\mathbf{P}_{ij} = \exp\left(\frac{\mathbf{f}_i}{\varepsilon}\right) \exp\left(-\frac{\mathbf{C}_{ij}}{\varepsilon}\right) \exp\left(\frac{\mathbf{g}_j}{\varepsilon}\right) = \mathbf{u}_i \mathbf{K}_{ij} \mathbf{v}_j, \quad (8)$$

where $\mathbf{u}_i = \exp(\frac{\mathbf{f}_i}{\varepsilon})$, $\mathbf{v}_j = \exp(\frac{\mathbf{g}_j}{\varepsilon})$, and $\mathbf{K}_{ij} = \exp(-\frac{\mathbf{C}_{ij}}{\varepsilon})$, respectively. $\square$

Remark. Note that computation of $\mathbf{P}_{ij}$ are effectively in the exponential domain. If we choose to find optimal solution for $\mathbf{P}$ by updating $\mathbf{u}$ and $\mathbf{v}$, which would grow exponentially in $\mathbf{f}$ and $\mathbf{g}$. This is indeed the case for the vanilla Sinkhorn Algorithm, and thus it has numeric instability issue which need to be overcome by the “log-stabilization” technique. The idea is simple: just move all computation to the log-domain (see Section 3.3 for details).

3 Sinkhorn Algorithms

In this section, I will discuss three different versions of Sinkhorn algorithms.

3.1 Vanilla Sinkhorn Algorithm

Given the optimal solution to the Sinkhorn problem in Equation (6), with the constraints in Equation (1), we have

$$\text{diag } \mathbf{u} \cdot \mathbf{K} \cdot \text{diag } \mathbf{v} \cdot \mathbf{1}_N = \mathbf{a} \quad \text{and} \quad \text{diag } \mathbf{v} \cdot \mathbf{K}^\top \cdot \text{diag } \mathbf{u} \cdot \mathbf{1}_M = \mathbf{b}. \quad (9)$$

Since we also have $\text{diag } \mathbf{v} \cdot \mathbf{1}_N = \mathbf{v}$ and $\text{diag } \mathbf{u} \cdot \mathbf{1}_M = \mathbf{u}$, we have that

$$\mathbf{u} \odot (\mathbf{K}\mathbf{v}) = \mathbf{a} \quad \text{and} \quad \mathbf{v} \odot (\mathbf{K}^\top \mathbf{u}) = \mathbf{b}. \quad (10)$$

Equivalently, we can also write the solution as

$$\mathbf{u} = \mathbf{a} \oslash (\mathbf{K}\mathbf{v}) \quad \text{and} \quad \mathbf{v} = \mathbf{b} \oslash (\mathbf{K}^\top \mathbf{u}). \quad (11)$$

In the above equations, $\odot$ and $\oslash$ refer to element-wise (Hadamard) multiplication and division, respectively. Therefore, we can leverage Equation (11) to update $\mathbf{u}$ and $\mathbf{v}$ till convergence and conclude with optimal coupling $\mathbf{P}$ by Equation (6), as we also know that the solution is unique as shown in Proposition 3.

Update 3.1 (Updating Equations for Vanilla Sinkhorn Algorithm). *At any iteration $\ell = 1, \dots, L$ with initialized $\mathbf{v}^{(0)} = \mathbf{1}_N$, we have*

$$\mathbf{u}^{(\ell)} \equiv \mathbf{a} \oslash (\mathbf{K}\mathbf{v}^{(\ell-1)}) \quad \text{and} \quad \mathbf{v}^{(\ell)} \equiv \mathbf{b} \oslash (\mathbf{K}^\top \mathbf{u}^{(\ell)}). \quad (12)$$

Remark. *In Update 3.1, the choice of $\mathbf{v}^{(0)}$ is rather arbitrary, as long as it is not $\mathbf{0}$. The most common choice is to consider the vectors of ones, i.e. $\mathbf{1}_N$. Changing the initialization vector will result in different solutions to $\mathbf{u}$ and $\mathbf{v}$, since they are only defined up to a multiplicative constant. But $\mathbf{u}$ and $\mathbf{v}$ will converge regardless of the initialization, and result in the same optimal coupling matrix $\mathbf{P}$ upon convergence. See Peyré and Cuturi (2019, p.82) for discussion.*

After L iterations, the scaling variables $\mathbf{u}^{(L)}$ and $\mathbf{v}^{(L)}$ can be used to compute the optimal coupling matrix $\mathbf{P}^{(L)} = \text{diag } \mathbf{u}^{(L)} \cdot \mathbf{K} \cdot \text{diag } \mathbf{v}^{(L)}$. From Update 3.1, we can arrive at the vanilla Sinkhorn algorithm (Algorithm 3.1).

Algorithm 3.1 Vanilla Sinkhorn Algorithm**Input:** $\mathbf{a} \in \Sigma_M$, $\mathbf{b} \in \Sigma_N$, $\mathbf{C} \in \mathbb{R}^{M \times N}$, $\varepsilon > 0$.**Initialize:** $\mathbf{u} = \mathbf{1}_M$, $\mathbf{v} = \mathbf{1}_N$.

- 1: $\mathbf{K} = \exp(-\frac{\mathbf{C}}{\varepsilon})$
- 2: **while** Not Convergence **do**
- 3: $\mathbf{u} = \mathbf{a} \oslash (\mathbf{K}\mathbf{v})$
- 4: $\mathbf{v} = \mathbf{b} \oslash (\mathbf{K}^\top \mathbf{u})$
- 5: **end while**
- 6: $\mathbf{P} = \text{diag } \mathbf{u} \cdot \mathbf{K} \cdot \text{diag } \mathbf{v}$

Output: Optimal Coupling $\mathbf{P}$

Remark (Convergence Condition). *Since we have Equation (10), we can use those equations to check if the scaling variables $\mathbf{u}$ and $\mathbf{v}$ at the current iteration could approximate $\mathbf{a}$ and $\mathbf{b}$ well. That is, for some small $\rho > 0$ the algorithm terminates at iteration L , such that*

$$\|\mathbf{u}^{(L)} \odot (\mathbf{K}\mathbf{v}^{(L)}) - \mathbf{a}\|_2 \leq \rho \quad \text{and} \quad \|\mathbf{v}^{(L)} \odot (\mathbf{K}^\top \mathbf{u}^{(L)}) - \mathbf{b}\|_2 \leq \rho$$

If the norms are smaller than some threshold ρ , we can then terminate the program.

3.2 Parallel Sinkhorn Algorithm

Note that in the Vanilla Sinkhorn Algorithm, the computation are carried out by vector-vector element-wise computation, i.e., $\mathbf{u}, \mathbf{a}$, and $\mathbf{K}\mathbf{v} \in \mathbb{R}^M$ and $\mathbf{v}, \mathbf{b}$, and $\mathbf{K}^\top \mathbf{u} \in \mathbb{R}^N$. That is, if we have a single source $\mathbf{a}$ and a single target $\mathbf{b}$, we are computing the “one-to-one” transportation. However, sometimes we need to calculate one-to-many, many-to-one, or many-to-many transport and this can be easily computed in a parallelized fashion. All we need to do is to define the matrices $\mathbf{A}$ and $\mathbf{B}$ instead of the vectors $\mathbf{a}$ and $\mathbf{b}$, and all that remains is to carry out the matrix computation instead of the vector ones.

Definition 4. *For some $S \in \mathbb{N}_+$:*

- *Suppose we have $\mathbf{a} \in \Sigma_M$ and $\mathbf{b}_1, \dots, \mathbf{b}_S \in \Sigma_N$. Then we can define*

$$\mathbf{A} = [\mathbf{a}, \dots, \mathbf{a}] \in \mathbb{R}^{M \times S} \quad \text{and} \quad \mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_S] \in \mathbb{R}^{N \times S}.$$

- *Suppose we have $\mathbf{a}_1, \dots, \mathbf{a}_S \in \Sigma_M$ and $\mathbf{b} \in \Sigma_N$. Then we can define*

$$\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_S] \in \mathbb{R}^{M \times S} \quad \text{and} \quad \mathbf{B} = [\mathbf{b}, \dots, \mathbf{b}] \in \mathbb{R}^{N \times S}.$$

- *Suppose we have $\mathbf{a}_1, \dots, \mathbf{a}_S \in \Sigma_M$ and $\mathbf{b}_1, \dots, \mathbf{b}_S \in \Sigma_N$. Then we can define*

$$\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_S] \in \mathbb{R}^{M \times S} \quad \text{and} \quad \mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_S] \in \mathbb{R}^{N \times S}.$$

Remark. It should be fairly obvious that the first two scenarios are special cases of the third, i.e. many-to-many transport. In the many-to-many case, the number of columns in $\mathbf{A}$ should be equal to that of $\mathbf{B}$; whereas in one-to-many or many-to-one cases, we need to duplicate the single vector S times, as if we had a S -column matrix to begin with. In many modern linear algebra packages, this can be achieved automatically by “broadcasting,”⁹ without the need of actually doing the duplication of the column vectors. Or in terms of matrix computation, for example, $\mathbf{B} = [\mathbf{b}, \dots, \mathbf{b}] = \mathbf{1}_S^\top \otimes \mathbf{b} = \mathbf{b} \cdot \mathbf{1}_S^\top$. This will come into handy in Section 5.

Therefore, we can have the matrix version¹⁰ of Equation (11) with scaling variable $\mathbf{U} \in \mathbb{R}^{M \times S}$ and $\mathbf{V} \in \mathbb{R}^{N \times S}$,

$$\mathbf{U} = \mathbf{A} \oslash (\mathbf{K}\mathbf{V}) \quad \text{and} \quad \mathbf{V} = \mathbf{B} \oslash (\mathbf{K}^\top \mathbf{U}). \quad (13)$$

Hence, we could have the matrix version of the updating equations of Sinkhorn algorithm.

Update 3.2 (Updating Equations for Parallel Sinkhorn Algorithm). At any iteration $\ell = 1, \dots, L$ and initialized $\mathbf{V}^{(0)} \in \mathbb{R}^{N \times S}$, we have

$$\mathbf{U}^{(\ell)} = \mathbf{A} \oslash (\mathbf{K}\mathbf{V}^{(\ell-1)}) \quad \text{and} \quad \mathbf{V}^{(\ell)} = \mathbf{B} \oslash (\mathbf{K}^\top \mathbf{U}^{(\ell)}). \quad (14)$$

Remark. Since Update 3.2 only involves matrix operations, the Sinkhorn algorithm are intrinsically GPU friendly, as matrix operations are even more efficient on GPU than on CPU. This why some applications of OT in machine learning leverage GPU acceleration via neural network libraries (Flamary et al., 2021).

Algorithm 3.2 Parallel Sinkhorn Algorithm

Input: $\mathbf{A} \in \Sigma_{M \times S}$, $\mathbf{B} \in \Sigma_{N \times S}$, $\mathbf{C} \in \mathbb{R}^{M \times N}$, $\varepsilon > 0$.

Initialize: $\mathbf{U} = \mathbf{1}_{M \times S}$, $\mathbf{V} = \mathbf{1}_{N \times S}$.

1: $\mathbf{K} = \exp(-\frac{\mathbf{C}}{\varepsilon})$

2: **while** Not Convergence **do**

3: $\mathbf{U} = \frac{\mathbf{A}}{\mathbf{K}\mathbf{V}}$

4: $\mathbf{V} = \frac{\mathbf{B}}{\mathbf{K}^\top \mathbf{U}}$

5: **end while**

6: $\forall s, \mathbf{P}_s = \text{diag } \mathbf{U}_s \cdot \mathbf{K} \cdot \text{diag } \mathbf{V}_s$

Output: Optimal Coupling $\mathbf{P}_s, \forall s$

Remark (Convergence Condition). Similar to the termination condition in Section 3.1, we now have the matrix norms to check convergence. For some $\rho > 0$ the algorithm terminates at iteration L , such that

$$\|\mathbf{U}^{(L)} \oslash (\mathbf{K}\mathbf{V}^{(L)}) - \mathbf{A}\|_2 \leq \rho, \quad \text{and} \quad \|\mathbf{V}^{(L)} \oslash (\mathbf{K}^\top \mathbf{U}^{(L)}) - \mathbf{B}\|_2 \leq \rho.$$

⁹For example, Eigen (C++): https://eigen.tuxfamily.org/dox/group__TutorialReductionsVisitorsBroadcasting.html
 NumPy (Python): <https://numpy.org/doc/stable/user/basics.broadcasting.html>.

¹⁰Instead of computing the vector-vector computation in Equation (11) S times, we are computing a matrix-matrix product/division once. They are mathematically equivalent, but in practice, the latter would be much faster due to the implementation of linear algebra routines (e.g. BLAS). Hence, this simple extension is considered parallel since we process S columns of data in one batch.

3.3 Log-stabilized Sinkhorn

As can be seen in Equations (8) and (11), the updates on $\mathbf{u}$ and $\mathbf{v}$ are in the exponential domain. Therefore, if either $\mathbf{C}_{ij}$ is large or ε is small, the corresponding $\mathbf{K}_{ij}$ would be very large, and the subsequent updates will fail because of the Inf/NaN values introduced by the numeric overflow. This is commonly known as the numeric instability issue of the Sinkhorn algorithm (Peyré and Cuturi, 2019, Chapter 4.4).

To circumvent this instability issue, we could move the computation into the log-domain, therefore without the need for computation of the exponential function. To this end, instead of updating the $\mathbf{u}$ and $\mathbf{v}$ as in Update 3.1, we can consider the dual variables $\mathbf{f}$ and $\mathbf{g}$ in the Lagrangian in Equation (7). Since $\mathbf{u} = \exp\left(\frac{\mathbf{f}}{\varepsilon}\right)$ and $\mathbf{v} = \exp\left(\frac{\mathbf{g}}{\varepsilon}\right)$, we have

$$\mathbf{f} = \varepsilon \log(\mathbf{u}) \quad \text{and} \quad \mathbf{g} = \varepsilon \log(\mathbf{v}). \quad (15)$$

Then, we can also rewrite the elements in the coupling matrix $\mathbf{P}$ as

$$\mathbf{P}_{ij}^{(\ell)} = \mathbf{u}_i^{(\ell)} \mathbf{K}_{ij} \mathbf{v}_j^{(\ell)} = \exp\left(-\frac{\mathbf{C}_{ij} - \mathbf{f}_i^{(\ell)} - \mathbf{g}_j^{(\ell)}}{\varepsilon}\right)$$

and the matrix form of $\mathbf{P}$ expressed in $\mathbf{f}$ and $\mathbf{g}$ is

$$\mathbf{P}^{(\ell)} = \exp\left(-\frac{\mathbf{C} - \mathbf{f}^{(\ell)} \cdot \mathbf{1}_N^\top - \mathbf{1}_M \cdot \mathbf{g}^{(\ell)\top}}{\varepsilon}\right) = \exp\left(-\frac{\mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell)})}{\varepsilon}\right), \quad (16)$$

where we define

$$\mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell)}) = \mathbf{C} - \mathbf{f}^{(\ell)} \cdot \mathbf{1}_N^\top - \mathbf{1}_M \cdot \mathbf{g}^{(\ell)\top}. \quad (17)$$

Note that $\mathbf{f}$ and $\mathbf{g}$ are dual variables in the Lagrangian in Equation (7). Then we can plug in $\mathbf{u}$ and $\mathbf{v}$ to get the following updating equations

$$\begin{aligned} \mathbf{f}^{(\ell)} &= \varepsilon \log \mathbf{u}^{(\ell)} = \varepsilon \log \mathbf{a} - \varepsilon \log(\mathbf{K} \mathbf{v}^{(\ell-1)}), \\ \mathbf{g}^{(\ell)} &= \varepsilon \log \mathbf{v}^{(\ell)} = \varepsilon \log \mathbf{b} - \varepsilon \log(\mathbf{K}^\top \mathbf{u}^{(\ell)}). \end{aligned} \quad (18)$$

Since $\mathbf{K} \mathbf{v}^{(\ell-1)} \in \mathbb{R}^m$, $\forall i$, the i -th element is

$$\begin{aligned} (\mathbf{K} \mathbf{v}^{(\ell-1)})_i &= \sum_j \mathbf{K}_{ij} \mathbf{v}_j^{(\ell-1)} \\ &= \sum_j \exp\left(-\frac{\mathbf{C}_{ij}}{\varepsilon}\right) \exp\left(\frac{\mathbf{g}_j^{(\ell-1)}}{\varepsilon}\right) \\ &= \left[\sum_j \exp\left(-\frac{\mathbf{C}_{ij} - \mathbf{f}_i^{(\ell-1)} - \mathbf{g}_j^{(\ell-1)}}{\varepsilon}\right) \right] \exp\left(-\frac{\mathbf{f}_i^{(\ell-1)}}{\varepsilon}\right), \end{aligned}$$

and

$$\begin{aligned}
-\varepsilon \log (\mathbf{K} \mathbf{v}^{(\ell-1)})_i &= -\varepsilon \log \sum_j \exp\left(-\frac{\mathbf{C}_{ij} - \mathbf{f}_i^{(\ell-1)} - \mathbf{g}_j^{(\ell-1)}}{\varepsilon}\right) + \mathbf{f}_i^{(\ell-1)} \\
&= -\varepsilon \log \left[\exp\left(-\frac{\mathbf{C} - \mathbf{f}^{(\ell-1)} \cdot \mathbb{1}_N^\top - \mathbb{1}_M \cdot (\mathbf{g}^{(\ell-1)})^\top}{\varepsilon}\right) \cdot \mathbb{1}_N \right]_i + \mathbf{f}_i^{(\ell-1)} \\
&= -\varepsilon \log \left[\exp\left(-\frac{\mathbf{R}(\mathbf{f}^{(\ell-1)}, \mathbf{g}^{(\ell-1)})}{\varepsilon}\right) \cdot \mathbb{1}_N \right]_i + \mathbf{f}_i^{(\ell-1)}.
\end{aligned}$$

Therefore, we have

$$-\varepsilon \log (\mathbf{K} \mathbf{v}^{(\ell-1)}) = -\varepsilon \log \left[\exp\left(-\frac{\mathbf{R}(\mathbf{f}^{(\ell-1)}, \mathbf{g}^{(\ell-1)})}{\varepsilon}\right) \cdot \mathbb{1}_N \right] + \mathbf{f}^{(\ell-1)}. \quad (19)$$

And we have the updating equation for $\mathbf{f}^{(\ell)}$. If we follow similar steps, we can also get it for $\mathbf{g}^{(\ell)}$. Putting everything together,

$$\begin{aligned}
\mathbf{f}^{(\ell)} &= \mathbf{f}^{(\ell-1)} + \varepsilon \log \mathbf{a} - \varepsilon \log \left[\exp\left(-\frac{\mathbf{R}(\mathbf{f}^{(\ell-1)}, \mathbf{g}^{(\ell-1)})}{\varepsilon}\right) \cdot \mathbb{1}_N \right], \\
\mathbf{g}^{(\ell)} &= \mathbf{g}^{(\ell-1)} + \varepsilon \log \mathbf{b} - \varepsilon \log \left[\exp\left(-\frac{\mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell-1)})}{\varepsilon}\right)^\top \cdot \mathbb{1}_M \right].
\end{aligned} \quad (20)$$

Note that Equation (20) is essentially a “log-sum-exp” form, and we can apply the “log-sum-exp” trick¹¹ to make this equation numeric stable while computing it in practice.

To see this, first we need to write the *soft-minimum* function (Peyré and Cuturi, 2019, Remark 4.22), let $\mathbf{z}$ be a vector of length n ,

$$\min_{\varepsilon} \mathbf{z} = -\varepsilon \log \sum_i \exp\left(-\frac{\mathbf{z}_i}{\varepsilon}\right).$$

Notice that when $\mathbf{z}_i$ is negative but large in absolute value, the exponential and hence its sum will likely encounter numeric overflow, which makes this computation not numerically stable. To circumvent this, let us denote $\underline{z} = \min \mathbf{z}$ and notice that the *soft-minimum* is the same as

$$\min_{\varepsilon} \mathbf{z} = \underline{z} - \varepsilon \log \sum_i \exp\left(-\frac{\mathbf{z} - \underline{z}}{\varepsilon}\right). \quad (21)$$

This equivalent form guarantees that the maximum value of the vector before exponentiation is 0 and hence remains numerically stable. Then let us denote any matrix $\mathbf{Z} \in \mathbb{R}^{M \times N}$ and $-\log [\exp(-\frac{\mathbf{Z}}{\varepsilon}) \cdot \mathbb{1}_N]$ is equivalent to applying the *soft-minimum* function to each row of

¹¹<https://en.wikipedia.org/wiki/LogSumExp>

the matrix $\mathbf{Z}$. Similarly, $-\log \left[\exp \left(-\frac{\mathbf{Z}}{\varepsilon} \right)^\top \cdot \mathbf{1}_M \right]$ is equivalent to applying the *soft-minimum* function to each column of the matrix $\mathbf{Z}$. Let us define the *row-wise soft-minimum* and *column-wise soft-minimum* as in Equations (4.40) and (4.41) from Peyré and Cuturi (2019),

Update 3.3 (Updating Equations for Log-Stabilized Sinkhorn Algorithm). *At any iteration $\ell = 1, \dots, L$ and initialized $\mathbf{f}^{(0)}$ and $\mathbf{g}^{(0)}$, we have*

$$\begin{aligned} \mathbf{f}^{(\ell)} &= \mathbf{f}^{(\ell-1)} + \varepsilon \log \mathbf{a} + \text{Min}_\varepsilon^{\text{row}} \mathbf{R}(\mathbf{f}^{(\ell-1)}, \mathbf{g}^{(\ell-1)}), \\ \mathbf{g}^{(\ell)} &= \mathbf{g}^{(\ell-1)} + \varepsilon \log \mathbf{b} + \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell-1)}), \end{aligned}$$

where for a matrix $\mathbf{Z} \in \mathbb{R}^{M \times N}$ we have

$$\begin{aligned} \text{Min}_\varepsilon^{\text{row}}(\mathbf{Z}) &= \left(\min_\varepsilon (\mathbf{Z}_{i,j})_j \right)_i \in \mathbb{R}^M, \\ \text{Min}_\varepsilon^{\text{col}}(\mathbf{Z}) &= \left(\min_\varepsilon (\mathbf{Z}_{i,j})_i \right)_j \in \mathbb{R}^N. \end{aligned}$$

In other words,

$$\begin{aligned} \text{Min}_\varepsilon^{\text{row}}(\mathbf{Z}) &= \left[\min_\varepsilon \mathbf{Z}_{1,\cdot}, \dots, \min_\varepsilon \mathbf{Z}_{M,\cdot} \right]^\top, \\ \text{Min}_\varepsilon^{\text{col}}(\mathbf{Z}) &= \left[\min_\varepsilon \mathbf{Z}_{\cdot,1}, \dots, \min_\varepsilon \mathbf{Z}_{\cdot,N} \right]^\top, \end{aligned}$$

where $\mathbf{Z}_{i,\cdot}$ and $\mathbf{Z}_{\cdot,j}$ denote the i -th row and j -th column of $\mathbf{Z}$ respectively.

Thus, we have obtained the numerically stable version of the updating equations for $\mathbf{f}$ and $\mathbf{g}$.

Remark. *As mentioned in Section 3.1, the initialization for $\mathbf{u}$ and $\mathbf{v}$ are vectors of ones. This is to say that, the initializations of $\mathbf{f}$ and $\mathbf{g}$ are vectors of zeros, i.e. $\mathbf{f}^{(0)} = \mathbf{0}_M$ and $\mathbf{g}^{(0)} = \mathbf{0}_N$.*

Therefore, we can have the Log-Stabilized Sinkhorn algorithm.

Algorithm 3.3 Log-Stabilized Sinkhorn Algorithm

Input: $\mathbf{a} \in \Sigma_M$, $\mathbf{b} \in \Sigma_N$, $\mathbf{C} \in \mathbb{R}^{M \times N}$, $\varepsilon > 0$.

Initialize: $\mathbf{f} = \mathbf{0}_M$, $\mathbf{g} = \mathbf{0}_N$.

```

1: while Not Convergence do
2:   # update f
3:    $\mathbf{R} = \mathbf{C} - \mathbf{f} \cdot \mathbf{1}_N^\top - \mathbf{1}_M \cdot \mathbf{g}^\top$ 
4:    $\mathbf{f} = \mathbf{f} + \varepsilon \log \mathbf{a} + \text{Min}_\varepsilon^{\text{row}} \mathbf{R}$ 
5:
6:   # update g
7:    $\mathbf{R} = \mathbf{C} - \mathbf{f} \cdot \mathbf{1}_N^\top - \mathbf{1}_M \cdot \mathbf{g}^\top$ 
8:    $\mathbf{g} = \mathbf{g} + \varepsilon \log \mathbf{b} + \text{Min}_\varepsilon^{\text{col}} \mathbf{R}$ 
9: end while
10:  $\mathbf{P} = \exp \left( -\frac{\mathbf{C} - \mathbf{f} \cdot \mathbf{1}_N^\top - \mathbf{1}_M \cdot \mathbf{g}^\top}{\varepsilon} \right)$ 
```

Output: $\mathbf{P}$

Remark. The advantage of using log-stabilized Sinkhorn is to make sure the computation is stable for any arbitrary ε . The disadvantage, however, is that we cannot easily parallel its computation over several margins as in Section 3.2. Moreover, the computation of matrix $\mathbf{Q}$ involves exp and log at every iteration twice for both updating $\mathbf{f}$ and $\mathbf{g}$. Thus, the log-stabilized Sinkhorn is less efficient than Vanilla and Parallel Sinkhorn algorithms. This is the tradeoff we need to make if we need to make sure the computation is stable, especially if we want to use a very small regularization constant ε . As in Peyré and Cuturi (2019, Chapter 4.1), the Sinkhorn problem converges to original Kantorovich problem when ε is small, i.e. $\ell_{\mathbf{C}}^{\varepsilon}(\mathbf{a}, \mathbf{b}) \rightarrow \ell_{\mathbf{C}}(\mathbf{a}, \mathbf{b})$ as $\varepsilon \rightarrow 0$.

Remark (Convergence Condition). Recall that in Section 3.1, we can compare the marginals reconstructed by the scaling vectors versus the real marginals, and now we can do the same step albeit in the log domain. That is, for some $\rho > 0$ the algorithm terminates at iteration L , such that

$$\begin{aligned} \|\text{Min}_{\varepsilon}^{\text{row}} \mathbf{R}(\mathbf{f}^{(L)}, \mathbf{g}^{(L)}) - \log \mathbf{a}\|_2 &\leq \rho, \\ \|\text{Min}_{\varepsilon}^{\text{col}} \mathbf{R}(\mathbf{f}^{(L)}, \mathbf{g}^{(L)}) - \log \mathbf{b}\|_2 &\leq \rho. \end{aligned}$$

4 Sinkhorn Algorithms with Gradients

In this section, I will derive the gradients of the loss function with respect to its arguments for the algorithms described in Section 3. This is useful when one wishes to use regularized loss in Equation (3) as a loss function for training purpose in machine learning, for example, Wasserstein Dictionary Learning in Sections 5 to 7. Similar to the structure in Section 3, I will derive the gradients/jacobians for the three algorithms respectively.

4.1 Vanilla Sinkhorn

The optimal coupling matrix $\mathbf{P}^*$ from the Vanilla Sinkhorn algorithm is

$$\mathbf{P}^* = \text{diag } \mathbf{u}^{(L)} \cdot \mathbf{K} \cdot \text{diag } \mathbf{v}^{(L)}, \quad (22)$$

and the loss evaluated at the solution is $\ell_{\mathbf{C}}^{\varepsilon}(\mathbf{P}^*) = \langle \mathbf{P}^*, \mathbf{C} \rangle + \varepsilon \langle \mathbf{P}^*, \log \mathbf{P}^* - 1 \rangle$

Therefore, by Chain's rule, the Jacobian of Sinkhorn loss w.r.t. first argument $\mathbf{a}$ is

$$D_{\mathbf{a}} \ell_{\mathbf{C}}^{\varepsilon}(\mathbf{P}^*) = D_{\mathbf{P}^*} \ell_{\mathbf{C}}^{\varepsilon} \cdot D_{\mathbf{a}} \mathbf{P}^*. \quad (23)$$

Note that $\langle \mathbf{A}, \mathbf{B} \rangle = \mathbf{1}_M^{\top} \cdot (\mathbf{A} \odot \mathbf{B}) \cdot \mathbf{1}_N$, for matrices $\mathbf{A}$ and $\mathbf{B}$ in $\mathbb{R}^{M \times N}$. To get the first term $D_{\mathbf{P}^*} \ell_{\mathbf{C}}^{\varepsilon}$, we need the differential as the following

$$\begin{aligned} d \text{vec } \ell_{\mathbf{C}}^{\varepsilon}(\mathbf{P}^*) &= d \text{vec } [\mathbf{1}_M^{\top} (\mathbf{P}^* \odot \mathbf{C}) \mathbf{1}_N] + \varepsilon d \text{vec } \{ \mathbf{1}_M^{\top} [\mathbf{P}^* \odot (\log \mathbf{P}^* - 1)] \mathbf{1}_N \} \\ &= d \left[(\mathbf{1}_N \otimes \mathbf{1}_M)^{\top} (\text{vec } \mathbf{C} \otimes \text{vec } \mathbf{P}^*) \right] + \varepsilon d \left\{ (\mathbf{1}_N \otimes \mathbf{1}_M)^{\top} [\text{vec } \mathbf{P}^* \otimes \text{vec } (\log \mathbf{P}^* - 1)] \right\} \\ &= \mathbf{1}_{MN}^{\top} (\text{vec } \mathbf{C} \otimes d \text{vec } \mathbf{P}^*) + \varepsilon \mathbf{1}_{MN}^{\top} \cdot d [\text{vec } \mathbf{P}^* \otimes \text{vec } (\log \mathbf{P}^* - 1)] \\ &= \mathbf{1}_{MN}^{\top} \cdot \text{diag } \text{vec } \mathbf{C} \cdot d \text{vec } \mathbf{P}^* + \varepsilon \mathbf{1}_{MN}^{\top} \cdot \text{diag } \text{vec } \log \mathbf{P}^* \cdot d \text{vec } \mathbf{P}^* \\ &= \mathbf{1}_{MN}^{\top} \cdot \text{diag } \text{vec } (\mathbf{C} + \varepsilon \log \mathbf{P}^*) \cdot d \text{vec } \mathbf{P}^*, \end{aligned}$$

and the Jacobian is

$$D_{\mathbf{P}^*} \ell_{\mathbf{C}}^\varepsilon = \mathbb{1}_{MN}^\top \cdot \text{diag vec} (\mathbf{C} + \varepsilon \log \mathbf{P}^*). \quad (24)$$

Note that this Jacobian matrix has only one row, since $(\mathbb{1}_N \otimes \mathbb{1}_M) = \mathbb{1}_{MN}$ has only one column. Thus, the gradient¹² $\nabla_{\mathbf{a}} \ell_{\mathbf{C}}^\varepsilon$, would be column vector of size M .

To get the second term, i.e. $D_{\mathbf{a}} \mathbf{P}^*$, we also need the differential of $\mathbf{P}^*$ in Equation (16) instead of Equation (22), as Equation (16) involves the dual variables $\mathbf{f}$ and $\mathbf{g}$, which will be useful when we derive the Jacobian matrices for the log-stabilized Sinkhorn algorithm in Section 3.3.

The differential is

$$\begin{aligned} d \text{vec } \mathbf{P}^* &= -\frac{1}{\varepsilon} \text{diag vec } \mathbf{P}^* \cdot d \text{vec} (\mathbf{C} - \mathbf{f}^{(L)} \cdot \mathbb{1}_N^\top - \mathbb{1}_M \cdot \mathbf{g}^{(L)\top}) \\ &= \frac{1}{\varepsilon} \text{diag vec } \mathbf{P}^* \cdot [d \text{vec} (\mathbf{f}^{(L)} \cdot \mathbb{1}_N^\top) + d \text{vec} (\mathbb{1}_m \cdot \mathbf{g}^{(L)\top})] \\ &= \frac{1}{\varepsilon} \text{diag vec } \mathbf{P}^* \cdot [(\mathbb{1}_N \otimes I_M) d\mathbf{f}^{(L)} + (I_N \otimes \mathbb{1}_M) d\mathbf{g}^{(L)}] \\ &= \text{diag vec } \mathbf{P}^* \cdot \left[(\mathbb{1}_N \otimes I_M) \cdot \text{diag} \frac{1}{\mathbf{u}^{(L)}} \cdot d\mathbf{u}^{(L)} + (I_N \otimes \mathbb{1}_M) \cdot \text{diag} \frac{1}{\mathbf{v}^{(L)}} \cdot d\mathbf{v}^{(L)} \right], \end{aligned} \quad (25)$$

since we also have $\mathbf{f} = \varepsilon \log \mathbf{u}$ and $\mathbf{g} = \varepsilon \log \mathbf{v}$, $d\mathbf{f} = \varepsilon \text{diag} \frac{1}{\mathbf{u}} \cdot d\mathbf{u}$ and $d\mathbf{g} = \varepsilon \text{diag} \frac{1}{\mathbf{v}} \cdot d\mathbf{v}$, respectively.

Therefore,

$$D_{\mathbf{a}} \mathbf{P}^* = \text{diag vec } \mathbf{P}^* \cdot \left[(\mathbb{1}_N \otimes I_M) \cdot \text{diag} \frac{1}{\mathbf{u}^{(L)}} \cdot D_{\mathbf{a}} \mathbf{u}^{(L)} + (I_N \otimes \mathbb{1}_M) \cdot \text{diag} \frac{1}{\mathbf{v}^{(L)}} \cdot D_{\mathbf{a}} \mathbf{v}^{(L)} \right]. \quad (26)$$

From Equation (26), all that remains to be shown are the formulae of $D_{\mathbf{a}} \mathbf{u}^{(L)}$ and $D_{\mathbf{a}} \mathbf{v}^{(L)}$, which can be obtained iteratively differentiating Update 3.1. Again, we need first to calculate

¹² $\nabla_{\mathbf{a}} \ell_{\mathbf{C}}^\varepsilon = (D_{\mathbf{a}} \ell_{\mathbf{C}}^\varepsilon)^\top$.

the differentials

$$\begin{aligned}
d\mathbf{u}^{(\ell)} &= d\left(\mathbf{a} \odot \frac{1}{\mathbf{K}\mathbf{v}^{(\ell-1)}}\right) \\
&= d\mathbf{a} \odot \frac{1}{\mathbf{K}\mathbf{v}^{(\ell-1)}} - \mathbf{a} \odot d\left(\frac{1}{\mathbf{K}\mathbf{v}^{(\ell-1)}}\right) \\
&= \text{diag}\left(\frac{1}{\mathbf{K}\mathbf{v}^{(\ell-1)}}\right) \cdot d\mathbf{a} - \text{diag} \frac{\mathbf{a}}{(\mathbf{K}\mathbf{v}^{(\ell-1)})^2} \cdot \mathbf{K} \cdot d\mathbf{v}^{(\ell-1)}, \\
d\mathbf{v}^{(\ell)} &= d\left(\mathbf{b} \odot \frac{1}{\mathbf{K}^\top \mathbf{u}^{(\ell)}}\right) \\
&= d\mathbf{b} \odot \frac{1}{\mathbf{K}^\top \mathbf{u}^{(\ell)}} + \mathbf{b} \odot d\left(\frac{1}{\mathbf{K}^\top \mathbf{u}^{(\ell)}}\right) \\
&= -\text{diag} \frac{\mathbf{b}}{(\mathbf{K}^\top \mathbf{u}^{(\ell)})^2} \cdot \mathbf{K}^\top \cdot d\mathbf{u}^{(\ell)}.
\end{aligned}$$

Therefore, the Jacobian updating equations are

Update 4.1 (Jacobian Updating Equations for Vanilla Sinkhorn Algorithm).

$$\begin{aligned}
D_{\mathbf{a}} \mathbf{u}^{(\ell)} &= \text{diag} \frac{1}{\mathbf{K}\mathbf{v}^{(\ell-1)}} - \text{diag} \frac{\mathbf{a}}{(\mathbf{K}\mathbf{v}^{(\ell-1)})^2} \cdot \mathbf{K} \cdot D_{\mathbf{a}} \mathbf{v}^{(\ell-1)}, \\
D_{\mathbf{a}} \mathbf{v}^{(\ell)} &= -\text{diag} \frac{\mathbf{b}}{(\mathbf{K}^\top \mathbf{u}^{(\ell)})^2} \cdot \mathbf{K}^\top \cdot D_{\mathbf{a}} \mathbf{u}^{(\ell)}.
\end{aligned} \tag{27}$$

Therefore, for the initialized $D_{\mathbf{a}} \mathbf{v}^{(0)} = \mathbf{0}_{N \times M}$, we can compute $D_{\mathbf{a}} \mathbf{u}^{(\ell)}$ and $D_{\mathbf{a}} \mathbf{v}^{(\ell)}$ for any ℓ , including L . Then we can finally arrive at the formula for the Jacobian

$$\begin{aligned}
D_{\mathbf{a}} \ell_{\mathbf{C}}^\varepsilon &= D_{\mathbf{P}^*} \ell_{\mathbf{C}}^\varepsilon \cdot D_{\mathbf{a}} \mathbf{P}^* \\
&= \mathbf{1}_{MN}^\top \cdot \text{diag} \text{vec}(\mathbf{C} + \varepsilon \log \mathbf{P}^*) \cdot \text{diag} \text{vec} \mathbf{P}^* \\
&\quad \cdot \left[(\mathbf{1}_N \otimes I_M) \cdot \text{diag} \frac{1}{\mathbf{u}^{(L)}} \cdot D_{\mathbf{a}} \mathbf{u}^{(L)} + (I_N \otimes \mathbf{1}_M) \cdot \text{diag} \frac{1}{\mathbf{v}^{(L)}} \cdot D_{\mathbf{a}} \mathbf{v}^{(L)} \right]
\end{aligned} \tag{28}$$

and the gradient $\nabla_{\mathbf{a}} \ell_{\mathbf{C}}^\varepsilon = \left[D_{\mathbf{a}} \ell_{\mathbf{C}}^\varepsilon \right]^\top$.

We can combine the Jacobian updates with the Sinkhorn updates in one loop, hence without the need to save intermediate Jacobian matrices, as in the case of Schmitz et al. (2018).

Algorithm 4.1 Vanilla Sinkhorn Algorithm with Gradient

Input: $\mathbf{a} \in \Sigma_M$, $\mathbf{b} \in \Sigma_N$, $\mathbf{C} \in \mathbb{R}^{M \times N}$, $\varepsilon > 0$.**Initialize:** $\mathbf{u} = \mathbf{1}_M$, $\mathbf{v} = \mathbf{1}_N$, $\mathbf{J}_{\mathbf{u}} = \mathbf{0}_{M \times M}$, $\mathbf{J}_{\mathbf{v}} = \mathbf{0}_{N \times M}$.

```

1:  $\mathbf{K} = \exp(-\frac{\mathbf{C}}{\varepsilon})$ 
2: while Not Convergence do
3:   # update u and Ju
4:    $\mathbf{u} = \mathbf{a} \oslash (\mathbf{K}\mathbf{v})$ 
5:    $\mathbf{J}_{\mathbf{u}} = \text{diag } \frac{1}{\mathbf{K}\mathbf{v}} - \text{diag } \frac{\mathbf{a}}{(\mathbf{K}\mathbf{v})^2} \cdot \mathbf{K} \cdot \mathbf{J}_{\mathbf{v}}$ 
6:
7:   # update v and Jv
8:    $\mathbf{v} = \mathbf{b} \oslash (\mathbf{K}^\top \mathbf{u})$ 
9:    $\mathbf{J}_{\mathbf{v}} = -\text{diag } \frac{\mathbf{b}}{(\mathbf{K}^\top \mathbf{u})^2} \cdot \mathbf{K}^\top \cdot \mathbf{J}_{\mathbf{u}}$ 
10: end while
11: # compute optimal coupling
12:  $\mathbf{P} = \text{diag } \mathbf{u} \cdot \mathbf{K} \cdot \text{diag } \mathbf{v}$ 
13: # compute gradient
14:  $\mathbf{J}_{\mathbf{P}} = \mathbf{1}_{MN}^\top \cdot \text{diag } \text{vec } (\mathbf{C} + \varepsilon \log \mathbf{P})$ 
15:  $\mathbf{J}_{\mathbf{a}} = \text{diag } \text{vec } \mathbf{P} \cdot [(\mathbf{1}_N \otimes I_M) \cdot \text{diag } \frac{1}{\mathbf{u}} \cdot \mathbf{J}_{\mathbf{u}} + (I_N \otimes \mathbf{1}_M) \cdot \text{diag } \frac{1}{\mathbf{v}} \cdot \mathbf{J}_{\mathbf{v}}]$ 
16:  $\nabla_{\mathbf{a}} = (\mathbf{J}_{\mathbf{P}} \cdot \mathbf{J}_{\mathbf{a}})^\top$ 
Output:  $\mathbf{P}$ ,  $\nabla_{\mathbf{a}}$ 

```

4.2 Parallel Sinkhorn

For the Parallel Sinkhorn algorithm, since we have S transport maps being computed at the same time, as supposed to the Vanilla Sinkhorn where we only compute one transport map at a time, we would have a loss vector of length S instead of a scalar. Therefore, without a proper way to aggregate the loss vector into a scalar, we cannot derive the gradient of the loss. We can, however, derive the Jacobian updating equations for the Parallel Sinkhorn algorithm, which is the focus of this subsection. With the Jacobians defined recursively, one can easily apply it to gradient computation, once the (scalar) loss function is properly defined.

A natural way to introduce an aggregation is to define the Wasserstein barycenter problem, where we can take the weighted sum of the Sinkhorn losses as the loss function for the minimization problem, but this discussion is deferred to Section 5.

Here, to derive the Jacobian updating equations, we can take the differentials

$$\begin{aligned}
d \operatorname{vec} \mathbf{U}^{(\ell)} &= d \left[\operatorname{vec} \mathbf{A} \odot \frac{1}{\operatorname{vec}(\mathbf{K}\mathbf{V}^{(\ell-1)})} \right] \\
&= d \operatorname{vec} \mathbf{A} \odot \frac{1}{\operatorname{vec}(\mathbf{K}\mathbf{V}^{(\ell-1)})} + \operatorname{vec} \mathbf{A} \odot d \frac{1}{\operatorname{vec}(\mathbf{K}\mathbf{V}^{(\ell-1)})} \\
&= \operatorname{diag} \operatorname{vec} \frac{1}{\mathbf{K}\mathbf{V}^{(\ell-1)}} \cdot d \operatorname{vec} \mathbf{A} - \operatorname{diag} \operatorname{vec} \mathbf{A} \cdot \operatorname{diag} \operatorname{vec} \frac{1}{(\mathbf{K}\mathbf{V}^{(\ell-1)})^2} \cdot d \operatorname{vec}(\mathbf{K}\mathbf{V}^{(\ell-1)}) \\
&= \operatorname{diag} \operatorname{vec} \frac{1}{\mathbf{K}\mathbf{V}^{(\ell-1)}} \cdot d \operatorname{vec} \mathbf{A} - \operatorname{diag} \operatorname{vec} \frac{\mathbf{A}}{(\mathbf{K}\mathbf{V}^{(\ell-1)})^2} \cdot (I_S \otimes \mathbf{K}) \cdot d \operatorname{vec} \mathbf{V}^{(\ell-1)},
\end{aligned} \tag{29}$$

and

$$\begin{aligned}
d \operatorname{vec} \mathbf{V}^{(\ell)} &= d \left[\operatorname{vec} \mathbf{B} \odot \frac{1}{\operatorname{vec}(\mathbf{K}^\top \mathbf{U}^{(\ell)})} \right] \\
&= \operatorname{vec} \mathbf{B} \odot d \frac{1}{\operatorname{vec}(\mathbf{K}^\top \mathbf{U}^{(\ell)})} \\
&= -\operatorname{diag} \operatorname{vec} \mathbf{B} \cdot \operatorname{diag} \operatorname{vec} \frac{1}{(\mathbf{K}^\top \mathbf{U}^{(\ell)})^2} \cdot d \operatorname{vec}(\mathbf{K}^\top \mathbf{U}^{(\ell)}) \\
&= -\operatorname{diag} \operatorname{vec} \frac{\mathbf{B}}{(\mathbf{K}^\top \mathbf{U}^{(\ell)})^2} \cdot (I_S \otimes \mathbf{K})^\top \cdot d \operatorname{vec} \mathbf{U}^{(\ell)}.
\end{aligned} \tag{30}$$

Therefore, we obtain the Jacobian updating equations.

Update 4.2 (Jacobian Updating Equations for Parallel Sinkhorn Algorithm).

$$\begin{aligned}
D_{\mathbf{A}} \mathbf{U}^{(\ell)} &= \operatorname{diag} \operatorname{vec} \frac{1}{\mathbf{K}\mathbf{V}^{(\ell-1)}} - \operatorname{diag} \operatorname{vec} \frac{\mathbf{A}}{(\mathbf{K}\mathbf{V}^{(\ell-1)})^2} \cdot (I_S \otimes \mathbf{K}) \cdot D_{\mathbf{A}} \mathbf{V}^{(\ell-1)}, \\
D_{\mathbf{A}} \mathbf{V}^{(\ell)} &= -\operatorname{diag} \operatorname{vec} \frac{\mathbf{B}}{(\mathbf{K}^\top \mathbf{U}^{(\ell)})^2} \cdot (I_S \otimes \mathbf{K})^\top \cdot D_{\mathbf{A}} \mathbf{U}^{(\ell)}.
\end{aligned} \tag{31}$$

Algorithm 4.2 Parallel Sinkhorn Algorithm with Jacobians**Input:** $\mathbf{A} \in \Sigma_{M \times S}$, $\mathbf{B} \in \Sigma_{N \times S}$, $\mathbf{C} \in \mathbb{R}^{M \times N}$, $\varepsilon > 0$.**Initialize:** $\mathbf{U} = \mathbf{1}_{M \times S}$, $\mathbf{V} = \mathbf{1}_{N \times S}$, $\mathbf{J}_{\mathbf{U}} = \mathbf{0}_{MS \times MS}$, $\mathbf{J}_{\mathbf{V}} = \mathbf{0}_{NS \times MS}$.

```

1:  $\mathbf{K} = \exp(-\frac{\mathbf{C}}{\varepsilon})$ 
2: while Not Convergence do
3:   # update U and JU
4:    $\mathbf{U} = \mathbf{A} \oslash (\mathbf{K}\mathbf{V})$ 
5:    $\mathbf{J}_{\mathbf{U}} = \text{diag vec } \frac{1}{\mathbf{K}\mathbf{V}} - \text{diag vec } \frac{\mathbf{A}}{(\mathbf{K}\mathbf{V})^2} \cdot (I_S \otimes \mathbf{K}) \cdot \mathbf{J}_{\mathbf{V}}$ 
6:
7:   # update V and JV
8:    $\mathbf{V} = \mathbf{B} \oslash (\mathbf{K}^\top \mathbf{U})$ 
9:    $\mathbf{J}_{\mathbf{V}} = -\text{diag vec } \frac{\mathbf{B}}{(\mathbf{K}^\top \mathbf{U})^2} \cdot (I_S \otimes \mathbf{K})^\top \cdot \mathbf{J}_{\mathbf{U}}$ 
10: end while
11:  $\forall s, \mathbf{P}_s = \text{diag}(\mathbf{U}_s) \cdot \mathbf{K} \cdot \text{diag}(\mathbf{V}_s)$ 
Output:  $\mathbf{P}_s$  for all  $s$ ,  $\mathbf{J}_{\mathbf{U}}$ ,  $\mathbf{J}_{\mathbf{V}}$ 

```

4.3 Log-Stabilized Sinkhorn

To derive the gradient of the log-stabilized Sinkhorn algorithm, we again use the Chain's rule to compute the Jacobian first as in Equation (23). We would thus have the same $D_{\mathbf{P}^*} \ell_{\mathbf{C}}^\varepsilon = \mathbf{1}_{MN}^\top \cdot \text{diag vec}(\mathbf{C} + \varepsilon \log \mathbf{P}^*)$, where $\mathbf{P}^* = \mathbf{P}^{(L)} = \exp\left(-\frac{\mathbf{R}(\mathbf{f}^{(L)}, \mathbf{g}^{(L)})}{\varepsilon}\right)$. To complete the derivation of the Jacobians, all we need to do is to obtain $D_{\mathbf{a}} \mathbf{P}^{(L)}$. Given Equations (16) and (17), we have

$$\begin{aligned}
d \text{ vec } \mathbf{P}^{(\ell)} &= d \text{ vec } \exp\left(-\frac{\mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell)})}{\varepsilon}\right) \\
&= d \exp\left(-\frac{\text{vec } \mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell)})}{\varepsilon}\right) \\
&= -\frac{1}{\varepsilon} \text{diag exp}\left(-\frac{\text{vec } \mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell)})}{\varepsilon}\right) \cdot d \text{ vec } \mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell)}) \\
&= \frac{1}{\varepsilon} \text{diag exp}\left(-\frac{\text{vec } \mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell)})}{\varepsilon}\right) \cdot [(\mathbf{1}_N \otimes I_M) d \mathbf{f}^{(\ell)} + (I_N \otimes \mathbf{1}_M) d \mathbf{g}^{(\ell)}],
\end{aligned}$$

and the Jacobian of $D \mathbf{P}^{(L)}$ is

$$D_{\mathbf{a}} \mathbf{P}^{(L)} = \frac{1}{\varepsilon} \text{diag exp}\left(-\frac{\text{vec } \mathbf{R}(\mathbf{f}^{(L)}, \mathbf{g}^{(L)})}{\varepsilon}\right) \cdot [(\mathbf{1}_N \otimes I_M) \cdot D_{\mathbf{a}} \mathbf{f}^{(L)} + (I_N \otimes \mathbf{1}_M) \cdot D_{\mathbf{a}} \mathbf{g}^{(L)}].$$

Then all we need is the updating equations for Jacobians $D_{\mathbf{a}} \mathbf{f}^{(\ell)}$ and $D_{\mathbf{a}} \mathbf{g}^{(\ell)}$. And those are given by the following updating equations with Chain's rule.

Update 4.3 (Jacobian Updating Equations for Log-Stabilized Sinkhorn Algorithm).

$$\begin{aligned} D_{\mathbf{a}} \mathbf{f}^{(\ell)} &= D_{\mathbf{a}} \mathbf{f}^{(\ell-1)} + \varepsilon \operatorname{diag} \frac{1}{\mathbf{a}} + D_{\mathbf{R}} \operatorname{Min}_{\varepsilon}^{\text{row}} \mathbf{R}(\mathbf{f}^{(\ell-1)}, \mathbf{g}^{(\ell-1)}) \cdot D_{\mathbf{a}} \mathbf{R}(\mathbf{f}^{(\ell-1)}, \mathbf{g}^{(\ell-1)}), \\ D_{\mathbf{a}} \mathbf{g}^{(\ell)} &= D_{\mathbf{a}} \mathbf{g}^{(\ell-1)} + D_{\mathbf{R}} \operatorname{Min}_{\varepsilon}^{\text{col}} \mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell-1)}) \cdot D_{\mathbf{a}} \mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell-1)}). \end{aligned}$$

From Equation (17) we have

$$\operatorname{vec} \mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell)}) = \operatorname{vec} \mathbf{C} - (\mathbb{1}_N \otimes I_M) \cdot \mathbf{f}^{(\ell)} - (I_N \otimes \mathbb{1}_M) \cdot \mathbf{g}^{(\ell)},$$

and obtain its differential w.r.t. $\mathbf{a}$

$$d \operatorname{vec} \mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell)}) = -(\mathbb{1}_N \otimes I_M) \cdot d \mathbf{f}^{(\ell)} - (I_N \otimes \mathbb{1}_M) \cdot d \mathbf{g}^{(\ell)}.$$

The two Jacobians are thus

$$\begin{aligned} D_{\mathbf{a}} \mathbf{R}(\mathbf{f}^{(\ell-1)}, \mathbf{g}^{(\ell-1)}) &= -(\mathbb{1}_N \otimes I_M) \cdot D_{\mathbf{a}} \mathbf{f}^{(\ell-1)} - (I_N \otimes \mathbb{1}_M) \cdot D_{\mathbf{a}} \mathbf{g}^{(\ell-1)}, \\ D_{\mathbf{a}} \mathbf{R}(\mathbf{f}^{(\ell)}, \mathbf{g}^{(\ell-1)}) &= -(\mathbb{1}_N \otimes I_M) \cdot D_{\mathbf{a}} \mathbf{f}^{(\ell)} - (I_N \otimes \mathbb{1}_M) \cdot D_{\mathbf{a}} \mathbf{g}^{(\ell-1)}. \end{aligned} \tag{32}$$

To complete the derivation of the Jacobians $D_{\mathbf{a}} \mathbf{f}^{(\ell)}$ and $D_{\mathbf{a}} \mathbf{g}^{(\ell)}$, we need the differential of the *soft-minimum* functions. From Lemma 9, we have that for a vector $\mathbf{z} \in \mathbb{R}^n$

$$\frac{\partial \min_{\varepsilon} \mathbf{z}}{\partial \mathbf{z}_i} = \frac{1}{\sum_{j=1}^n \exp\left(-\frac{\mathbf{z}_j - \mathbf{z}}{\varepsilon}\right)} \cdot \exp\left(-\frac{\mathbf{z}_i - \mathbf{z}}{\varepsilon}\right),$$

and for $h = 1, \dots, M$, $i = 1, \dots, M$, $j = 1, \dots, N$, and $\mathbf{Z} \in \mathbb{R}^{M \times N}$ we have

$$\frac{\partial \min_{\varepsilon} \mathbf{Z}_{h,\cdot}}{\partial \mathbf{Z}_{i,j}} = \begin{cases} \frac{1}{\mathbb{1}_N^{\top} \cdot \exp\left(-\frac{\mathbf{Z}_{h,\cdot} - \min \mathbf{Z}_{h,\cdot}}{\varepsilon}\right)} \cdot \exp\left(-\frac{\mathbf{Z}_{h,\cdot} - \min \mathbf{Z}_{h,\cdot}}{\varepsilon}\right), & \text{if } i = h, \\ 0, & \text{otherwise.} \end{cases}$$

Then we have

$$D_{\mathbf{Z}} \min_{\varepsilon} \mathbf{Z}_{h,\cdot} = \left[\operatorname{vec} \left(\left[\frac{\partial \min_{\varepsilon} \mathbf{Z}_{h,\cdot}}{\partial \mathbf{Z}_{i,j}} \right]_{i,j} \right) \right]^{\top},$$

and

$$D_{\mathbf{Z}} \operatorname{Min}_{\varepsilon}^{\text{row}} \mathbf{Z} = D_{\mathbf{Z}} \begin{bmatrix} \min_{\varepsilon} \mathbf{Z}_{1,\cdot} \\ \dots \\ \min_{\varepsilon} \mathbf{Z}_{M,\cdot} \end{bmatrix} = \begin{bmatrix} D_{\mathbf{Z}} \min_{\varepsilon} \mathbf{Z}_{1,\cdot} \\ \dots \\ D_{\mathbf{Z}} \min_{\varepsilon} \mathbf{Z}_{M,\cdot} \end{bmatrix}. \tag{33}$$

Similarly, we can arrive at the Jacobian for $\operatorname{Min}_{\varepsilon}^{\text{col}}$,

$$D_{\mathbf{Z}} \operatorname{Min}_{\varepsilon}^{\text{col}} \mathbf{Z} = D_{\mathbf{Z}} \begin{bmatrix} \min_{\varepsilon} \mathbf{Z}_{\cdot,1} \\ \dots \\ \min_{\varepsilon} \mathbf{Z}_{\cdot,N} \end{bmatrix} = \begin{bmatrix} D_{\mathbf{Z}} \min_{\varepsilon} \mathbf{Z}_{\cdot,1} \\ \dots \\ D_{\mathbf{Z}} \min_{\varepsilon} \mathbf{Z}_{\cdot,N} \end{bmatrix}, \tag{34}$$

where

$$\frac{\partial \min_{\varepsilon} \mathbf{Z}_{\cdot,t}}{\partial \mathbf{Z}_{i,j}} = \begin{cases} \frac{1}{\mathbb{1}_M^{\top} \cdot \exp\left(-\frac{\mathbf{Z}_{\cdot,t} - \min \mathbf{Z}_{\cdot,t}}{\varepsilon}\right)} \cdot \exp\left(-\frac{\mathbf{Z}_{\cdot,t} - \min \mathbf{Z}_{\cdot,t}}{\varepsilon}\right), & \text{if } j = t, \\ 0, & \text{otherwise,} \end{cases}$$

and

$$D_{\mathbf{Z}} \min_{\varepsilon} \mathbf{Z}_{\cdot,t} = \left[\text{vec} \left(\left[\frac{\partial \min_{\varepsilon} \mathbf{Z}_{\cdot,t}}{\partial \mathbf{Z}_{i,j}} \right]_{i,j} \right) \right]^{\top}.$$

Algorithm 4.3 Log-Stabilized Sinkhorn with Gradient

Input: $\mathbf{a} \in \Sigma_M$, $\mathbf{b} \in \Sigma_N$, $\mathbf{C} \in \mathbb{R}^{M \times N}$, $\varepsilon > 0$.

Initialize: $\mathbf{f} = \mathbf{0}_M$, $\mathbf{g} = \mathbf{0}_N$, $\mathbf{J}_{\mathbf{f}} = \mathbf{0}_{M \times M}$, $\mathbf{J}_{\mathbf{g}} = \mathbf{0}_{N \times N}$.

```

1:  $\mathbf{K} = \text{Commutation}(M, N)$  # commutation matrix K
2: while Not Convergence do
3:   # update f and Jf
4:    $\mathbf{R} = \mathbf{C} - \mathbf{f} \cdot \mathbb{1}_N^{\top} - \mathbb{1}_M \cdot \mathbf{g}^{\top}$ 
5:    $\mathbf{f} = \mathbf{f} + \varepsilon \log \mathbf{a} + \text{Min}_{\varepsilon}^{\text{row}} \mathbf{R}$ 
6:    $\mathbf{J}_{\mathbf{R}}$  from Equation (33)
7:    $\mathbf{J}_{\mathbf{f}} = \mathbf{J}_{\mathbf{f}} + \varepsilon \text{diag} \frac{1}{\mathbf{a}} - \mathbf{J}_{\mathbf{R}} \cdot [(\mathbb{1}_N \otimes I_M) \cdot \mathbf{J}_{\mathbf{f}} + (I_N \otimes \mathbb{1}_M) \cdot \mathbf{J}_{\mathbf{g}}]$ 
8:
9:   # update g and Jg
10:   $\mathbf{R} = \mathbf{C} - \mathbf{f} \cdot \mathbb{1}_N^{\top} - \mathbb{1}_M \cdot \mathbf{g}^{\top}$ 
11:   $\mathbf{g} = \mathbf{g} + \varepsilon \log \mathbf{b} + \text{Min}_{\varepsilon}^{\text{col}} \mathbf{R}$ 
12:   $\mathbf{J}_{\mathbf{R}}$  from Equation (34)
13:   $\mathbf{J}_{\mathbf{g}} = \mathbf{J}_{\mathbf{g}} - \mathbf{J}_{\mathbf{R}} \cdot [(\mathbb{1}_N \otimes I_M) \cdot \mathbf{J}_{\mathbf{f}} + (I_N \otimes \mathbb{1}_M) \cdot \mathbf{J}_{\mathbf{g}}]$ 
14: end while
15:  $\mathbf{P} = \exp\left(-\frac{\mathbf{C} - \mathbf{f} \cdot \mathbb{1}_N^{\top} - \mathbb{1}_M \cdot \mathbf{g}^{\top}}{\varepsilon}\right)$  # optimal coupling
16:  $\mathbf{J}_{\mathbf{P}} = \mathbb{1}_{MN}^{\top} \cdot \text{diag} \text{vec}(\mathbf{C} + \varepsilon \log \mathbf{P})$ 
17:  $\mathbf{J}_{\mathbf{a}} = \frac{1}{\varepsilon} \text{diag} \text{vec} \mathbf{P} \cdot [(\mathbb{1}_N \otimes I_M) \cdot \mathbf{J}_{\mathbf{f}} + (I_N \otimes \mathbb{1}_M) \cdot \mathbf{J}_{\mathbf{g}}]$ 
18:  $\nabla_{\mathbf{a}} = (\mathbf{J}_{\mathbf{P}} \cdot \mathbf{J}_{\mathbf{a}})^{\top}$ 

```

Output: $\mathbf{P}$, $\nabla_{\mathbf{a}}$.

5 Wasserstein Barycenter

As is often in the case in practice, one wants to compute the “mean” or “center” given a sequence of points. This is useful when we want to find a unknown center as an estimate to the group of points (usually data). Suppose we have $(\mathbf{x}_s)_{s=1}^S \in \mathcal{X}^S$ in a metric space $(\mathcal{X}, d)$, where $d(\cdot)$ is a metric defined on $\mathcal{X}$, then we have the following problem

$$\min_{\mathbf{x} \in \mathcal{X}} \sum_s^S \mathbf{w}_s d(x, x_s)^p,$$

for a weight vector $\mathbf{w} \in \Sigma_S$ and some power p but is often set to 2. This is often called “Fréchet” or “Karcher” mean¹³.

5.1 Parallel Algorithm

We can then state the Wasserstein Barycenter problem, similar to the idea of “Fréchet mean” as a weighted sum of the individual distances/losses. Consider the loss function in Equation (2), a weight vector $\mathbf{w} \in \Sigma_S$, and a sequence of discrete densities $\{\mathbf{a}_s\}_{s=1}^S$, where $a_s \in \Sigma_{M_s}$ and M_s is the length of a_s , a Wasserstein barycenter is computed by the following minimization problem

$$\arg \min_{\mathbf{b} \in \Sigma_N} \sum_{s=1}^S \mathbf{w}_s \ell_{\mathbf{C}_s}(\mathbf{a}_s, \mathbf{b}),$$

where the cost matrix $\mathbf{C}_s \in \mathbb{R}^{M_s \times N}$ specifies the cost for transporting between the probability vectors $\mathbf{a}_s$ and $\mathbf{b}$. Of course, this problem can be approximated by the entropic regularized version by simply swapping the loss function by Equation (3). Thus, we have the entropic regularized Wasserstein barycenter problem

$$\arg \min_{\mathbf{b} \in \Sigma_N} \sum_{s=1}^S \mathbf{w}_s \ell_{\mathbf{C}_s}^\varepsilon(\mathbf{a}_s, \mathbf{b}). \quad (35)$$

$\mathbf{b}$ is therefore the unknown computed barycenter as the “mean” of the atoms $\mathbf{a}_1, \dots, \mathbf{a}_S$. By rewriting the problem in Equation (35) as (weighted) Kullback-Leibler (KL) projection problem (Benamou et al., 2015; Schmitz et al., 2018),

$$\min_{(\mathbf{P}_s)_s} \left\{ \sum_s w_s \varepsilon \mathbf{KL}(\mathbf{P}_s | \mathbf{K}_s) : \mathbf{P}_s \cdot \mathbf{1}_N = \mathbf{a}_s, \mathbf{P}_1^\top \cdot \mathbf{1}_{M_1} = \dots = \mathbf{P}_S^\top \cdot \mathbf{1}_{M_S} = \mathbf{b} \right\},$$

we will arrive at the Sinkhorn-like updates:

$$\begin{aligned} \mathbf{u}_s^{(\ell)} &= \frac{\mathbf{a}_s}{\mathbf{K}_s \mathbf{v}_s^{(\ell-1)}}, \\ \mathbf{b}^{(\ell)} &= \Pi_{s=1}^S \left(\mathbf{K}_s^\top \mathbf{u}_s^{(\ell)} \right)^{\mathbf{w}_s}, \\ \mathbf{v}_s^{(\ell)} &= \frac{\mathbf{b}^{(\ell)}}{\mathbf{K}_s^\top \mathbf{u}_s^{(\ell)}}, \end{aligned} \quad (36)$$

where $\mathbf{K}_s = \exp\left(-\frac{\mathbf{C}_s}{\varepsilon}\right)$, and $\mathbf{u}_s, \mathbf{v}_s$ are scaling vectors for $s = 1, \dots, S$. Also, $\dot{\cdot}$ and $(\cdot)^{(\cdot)}$ are to be understood as element-wise operations.

In practical applications, however, we usually have $M_1 = \dots = M_S = M$ and $\mathbf{K} = \mathbf{K}_1 = \dots = \mathbf{K}_S$. We can thus rewrite Equation (36) similar to Section 3.2 as the following updating equations. In some applications, for example Wasserstein Dictionary Learning (Schmitz et al., 2018), even $M = N$, since both “source” densities and “target” densities are token distributions of same length.

¹³Or Riemannian center of mass. See Karcher (2014) for a historical account on the naming.

Update 5.1 (Updating Equations for Parallel Wasserstein Barycenter Algorithm).

$$\begin{aligned} \mathbf{U}^{(\ell)} &= \frac{\mathbf{A}}{\mathbf{K}\mathbf{V}^{(\ell-1)}}, \\ \mathbf{b}^{(\ell)} &= \Pi_{\text{row}} \left(\mathbf{K}^\top \mathbf{U}^{(\ell)} \right)^{\mathbb{1}_N \cdot \mathbf{w}^\top}, \\ \mathbf{V}^{(\ell)} &= \frac{\mathbf{b}^{(\ell)} \cdot \mathbb{1}_S^\top}{\mathbf{K}^\top \mathbf{U}^{(\ell)}}, \end{aligned} \tag{37}$$

where $\mathbf{A} = [\mathbf{a}, \dots, \mathbf{a}] \in \mathbb{R}^{M \times S}$ and the product operator $\Pi_{\text{row}}(\cdot)$ refers to the row-wise product on the matrix.

The row-wise product will reduce the matrix into a length N vector, and the power operation in the second line refers to the element-wise power operation. The matrix $\mathbb{1}_N \cdot \mathbf{w}^\top$ simply populates the length S vector $\mathbf{w}$ into an $N \times S$ matrix, which can also be seen as applying each element of the $\mathbf{w}$ vector to the base matrix column-wise.

Algorithm 5.1 Parallel Wasserstein Barycenter Algorithm

Input: $\mathbf{A} \in \Sigma_{M \times S}$, $\mathbf{C} \in \mathbb{R}^{M \times N}$, $\mathbf{w} \in \Sigma_S$, $\varepsilon > 0$.

Initialize: $\mathbf{U} = \mathbb{1}_{M \times S}$, $\mathbf{V} = \mathbb{1}_{N \times S}$, $\mathbf{b} = \mathbf{0}_N$.

- 1: $\mathbf{K} = \exp(-\frac{\mathbf{C}}{\varepsilon})$
- 2: **while** Not Convergence **do**
- 3: $\mathbf{U} = \mathbf{A} \oslash (\mathbf{K}\mathbf{V})$
- 4: $\mathbf{b} = \Pi_{\text{row}} \left(\mathbf{K}^\top \mathbf{U} \right)^{\mathbb{1}_N \cdot \mathbf{w}^\top}$
- 5: $\mathbf{V} = (\mathbf{b} \cdot \mathbb{1}_S^\top) \oslash (\mathbf{K}^\top \mathbf{U})$
- 6: **end while**

Output: $\mathbf{b}$

Remark. In Line 4 of Algorithm 5.1, the notation Π_s refers to row-wise product reduction as in Equation (37). In Line 5, the numerator $\mathbf{b}$ can be broadcast to be the same numerator of all columns of the denominator matrix as discussed in Section 3.2.

Remark (Convergence Condition). Similar to the Convergence Condition in Section 3.3, we can check the following condition for convergence. For some $\rho > 0$,

$$\|\mathbf{U}^{(\ell)} \odot (\mathbf{K}\mathbf{V}^{(\ell)}) - \mathbf{A}\|_2 \leq \rho.$$

5.2 Log-stabilized Algorithm

Similar to the discussion of Section 3.3, numerical instability is still an issue for Algorithm 5.1 due to the computation of the exponential function and its potential numeric overflow. Therefore, we would also need to stabilize the computation in the log-domain. In order to do so, we will also need to state the algorithms (updating equations) in terms of the dual variables $\mathbf{f}$ and $\mathbf{g}$.

Given Update 3.3 we have that for $s = 1, \dots, S$,

$$\begin{aligned}\mathbf{f}_s^{(\ell)} &= \mathbf{f}_s^{(\ell-1)} + \varepsilon \log \mathbf{a} + \text{Min}_\varepsilon^{\text{row}} \mathbf{R}(\mathbf{f}_s^{(\ell-1)}, \mathbf{g}_s^{(\ell-1)}), \\ \mathbf{g}_s^{(\ell)} &= \mathbf{g}_s^{(\ell-1)} + \varepsilon \log \mathbf{b}^{(\ell)} + \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{f}_s^{(\ell)}, \mathbf{g}_s^{(\ell-1)}).\end{aligned}$$

Let us define

$$\begin{aligned}\mathbf{F}^{(\ell)} &\equiv [\mathbf{f}_1^{(\ell)}, \dots, \mathbf{f}_S^{(\ell)}] = [\varepsilon \log \mathbf{u}_1^{(\ell)}, \dots, \varepsilon \log \mathbf{u}_S^{(\ell)}] = \varepsilon \log \mathbf{U}^{(\ell)} \in \mathbb{R}^{M \times S}, \\ \mathbf{G}^{(\ell)} &\equiv [\mathbf{g}_1^{(\ell)}, \dots, \mathbf{g}_S^{(\ell)}] = [\varepsilon \log \mathbf{v}_1^{(\ell)}, \dots, \varepsilon \log \mathbf{v}_S^{(\ell)}] = \varepsilon \log \mathbf{V}^{(\ell)} \in \mathbb{R}^{N \times S},\end{aligned}$$

and from Equation (13) we will also need $\log(\mathbf{KV}^{(\ell-1)})$ and $\log(\mathbf{K}^\top \mathbf{U}^{(\ell-1)})$. From Section 3.3 we have

$$\begin{aligned}\log(\mathbf{KV}_s^{(\ell-1)}) &= \log \left[\exp \left(-\frac{\mathbf{R}(\mathbf{f}_s^{(\ell-1)}, \mathbf{g}_s^{(\ell-1)})}{\varepsilon} \right) \cdot \mathbb{1}_N \right] - \frac{1}{\varepsilon} \mathbf{f}_s^{(\ell-1)}, \\ \log(\mathbf{K}^\top \mathbf{u}_s^{(\ell-1)}) &= \log \left[\exp \left(-\frac{\mathbf{R}(\mathbf{f}_s^{(\ell)}, \mathbf{g}_s^{(\ell-1)})}{\varepsilon} \right)^\top \cdot \mathbb{1}_M \right] - \frac{1}{\varepsilon} \mathbf{g}_s^{(\ell-1)}.\end{aligned}$$

Thus,

$$\begin{aligned}\mathbf{F}^{(\ell)} &= \varepsilon \log \mathbf{U}^{(\ell)} \\ &= \varepsilon \log \mathbf{A} - \varepsilon \log(\mathbf{KV}^{(\ell-1)}) \\ &= \mathbf{F}^{(\ell-1)} + \varepsilon \log \mathbf{A} + \left[\dots, -\varepsilon \log \left[\exp \left(-\frac{\mathbf{R}(\mathbf{f}_s^{(\ell-1)}, \mathbf{g}_s^{(\ell-1)})}{\varepsilon} \right) \cdot \mathbb{1}_N \right], \dots \right] \\ &= \mathbf{F}^{(\ell-1)} + \varepsilon \log \mathbf{A} + [\dots, \text{Min}_\varepsilon^{\text{row}} \mathbf{R}(\mathbf{f}_s^{(\ell-1)}, \mathbf{g}_s^{(\ell-1)}), \dots] \\ &= \mathbf{F}^{(\ell-1)} + \varepsilon \log \mathbf{A} + \text{Min}_\varepsilon^{\text{row}} \mathbf{R}(\mathbf{F}^{(\ell-1)}, \mathbf{G}^{(\ell-1)}), \\ \mathbf{G}^{(\ell)} &= \varepsilon \log \mathbf{V}^{(\ell)} \\ &= \varepsilon \log \mathbf{B}^{(\ell)} - \varepsilon \log(\mathbf{K}^\top \mathbf{U}^{(\ell-1)}) \\ &= \mathbf{G}^{(\ell-1)} + \varepsilon \log \mathbf{b}^{(\ell)} \cdot \mathbb{1}_S^\top + \left[\dots, -\varepsilon \log \left[\exp \left(-\frac{\mathbf{R}(\mathbf{f}_s^{(\ell)}, \mathbf{g}_s^{(\ell-1)})}{\varepsilon} \right)^\top \cdot \mathbb{1}_M \right], \dots \right] \\ &= \mathbf{G}^{(\ell-1)} + \varepsilon \log \mathbf{b}^{(\ell)} \cdot \mathbb{1}_S^\top + [\dots, \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{f}_s^{(\ell)}, \mathbf{g}_s^{(\ell-1)}), \dots] \\ &= \mathbf{G}^{(\ell-1)} + \varepsilon \log \mathbf{b}^{(\ell)} \cdot \mathbb{1}_S^\top + \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}),\end{aligned} \tag{38}$$

where we slightly abuse the notation that

$$\begin{aligned}
& \text{Min}_\varepsilon^{\text{row}} \mathbf{R}(\mathbf{F}^{(\ell-1)}, \mathbf{G}^{(\ell-1)}) \\
&= \left[\text{Min}_\varepsilon^{\text{row}} \mathbf{R}(\mathbf{f}_1^{(\ell-1)}, \mathbf{g}_1^{(\ell-1)}), \dots, \text{Min}_\varepsilon^{\text{row}} \mathbf{R}(\mathbf{f}_s^{(\ell-1)}, \mathbf{g}_s^{(\ell-1)}), \dots, \text{Min}_\varepsilon^{\text{row}} \mathbf{R}(\mathbf{f}_S^{(\ell-1)}, \mathbf{g}_S^{(\ell-1)}) \right],
\end{aligned} \tag{39}$$

$$\begin{aligned}
& \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}) \\
&= \left[\text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{f}_1^{(\ell)}, \mathbf{g}_1^{(\ell-1)}), \dots, \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{f}_s^{(\ell)}, \mathbf{g}_s^{(\ell-1)}), \dots, \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{f}_S^{(\ell)}, \mathbf{g}_S^{(\ell-1)}) \right].
\end{aligned} \tag{40}$$

Since

$$\begin{aligned}
\log(\mathbf{K}^\top \mathbf{U}^{(\ell)}) &= -\frac{1}{\varepsilon} \mathbf{G}^{(\ell-1)} + \left[\dots, \log \left[\exp \left(-\frac{\mathbf{R}(\mathbf{f}_s^{(\ell)}, \mathbf{g}_s^{(\ell-1)})}{\varepsilon} \right)^\top \cdot \mathbf{1}_M \right], \dots \right] \\
&= -\frac{1}{\varepsilon} \mathbf{G}^{(\ell-1)} - \frac{1}{\varepsilon} [\dots, \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{f}_s^{(\ell)}, \mathbf{g}_s^{(\ell-1)}), \dots] \\
&= -\frac{1}{\varepsilon} \mathbf{G}^{(\ell-1)} - \frac{1}{\varepsilon} \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}),
\end{aligned}$$

then

$$\begin{aligned}
\log \mathbf{b}^{(\ell)} &= \log \Pi_s(\mathbf{K}^\top \mathbf{u}_s^{(\ell)})^{\mathbf{w}_s} \\
&= \Sigma_s \log(\mathbf{K}^\top \mathbf{u}_s^{(\ell)})^{\mathbf{w}_s} \\
&= \Sigma_s \mathbf{w}_s \log(\mathbf{K}^\top \mathbf{u}_s^{(\ell)}) \\
&= \log(\mathbf{K}^\top \mathbf{U}^{(\ell)}) \cdot \mathbf{w} \\
&= -\frac{1}{\varepsilon} \mathbf{G}^{(\ell-1)} \cdot \mathbf{w} - \frac{1}{\varepsilon} \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}) \cdot \mathbf{w}.
\end{aligned} \tag{41}$$

Putting everything together, we have the updating equations

Update 5.2 (Updating Equations for the Log-Stabilized Wasserstein Barycenter Algorithm).

$$\begin{aligned}
\mathbf{F}^{(\ell)} &= \mathbf{F}^{(\ell-1)} + \varepsilon \log \mathbf{A} + \text{Min}_\varepsilon^{\text{row}} \mathbf{R}(\mathbf{F}^{(\ell-1)}, \mathbf{G}^{(\ell)}), \\
\log \mathbf{b}^{(\ell)} &= -\frac{1}{\varepsilon} \mathbf{G}^{(\ell-1)} \cdot \mathbf{w} - \frac{1}{\varepsilon} \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}) \cdot \mathbf{w}, \\
\mathbf{G}^{(\ell)} &= \mathbf{G}^{(\ell-1)} + \varepsilon \log \mathbf{b}^{(\ell)} \cdot \mathbf{1}_S^\top + \text{Min}_\varepsilon^{\text{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}).
\end{aligned}$$

Algorithm 5.2 Log-Stabilized Wasserstein Barycenter Algorithm**Input:** $\mathbf{A} \in \Sigma_{M \times S}$, $\mathbf{C} \in \mathbb{R}^{M \times N}$, $\mathbf{w} \in \Sigma_S$, $\varepsilon > 0$.**Initialize:** $\mathbf{F} = \mathbf{0}_{M \times S}$, $\mathbf{G}_{N \times S} = \mathbf{1}_{N \times S}$, $\mathbf{logb} = \mathbf{0}_N$.

```

1: while Not Convergence do
2:   # update F
3:    $\mathbf{R}_{min}^{row}$  from Equation (39)
4:    $\mathbf{F} = \mathbf{F} + \varepsilon \log \mathbf{A} + \mathbf{R}_{min}^{row}$ 
5:
6:   # update logb
7:    $\mathbf{R}_{min}^{col}$  from Equation (40)
8:    $\mathbf{logb} = -\frac{1}{\varepsilon} \mathbf{G} \cdot \mathbf{w} - \frac{1}{\varepsilon} \mathbf{R}_{min}^{col} \cdot \mathbf{w}$ 
9:
10:  # update G
11:   $\mathbf{G} = \mathbf{G} + \varepsilon \mathbf{logb} \cdot \mathbf{1}_S^\top + \mathbf{R}_{min}^{col}$ 
12: end while
Output:  $\mathbf{b} = \exp \log \mathbf{b}$ 

```

6 Wasserstein Barycenter with Jacobians

Similar to Section 4, we need to manually derive the Jacobian calculations for the Wasserstein Barycenter algorithm of Algorithms 5.1 and 5.2. Since we have two versions of algorithms, we will need to derive their Jacobians separately.

6.1 Jacobians for Parallel Algorithm

As the updating equations for $\mathbf{U}^{(\ell)}$ and $\mathbf{V}^{(\ell)}$ are identical to that of Update 3.2, we would have almost identical differentials as in Section 4.2, with the slight difference that the differential of $\mathbf{V}^{(\ell)}$ now also depends on $\mathbf{b}^{(\ell)}$ and its differential,

$$\begin{aligned}
 d \operatorname{vec} \mathbf{U}^{(\ell)} &= \operatorname{diag} \operatorname{vec} \frac{1}{\mathbf{K} \mathbf{V}^{(\ell-1)}} \cdot d \operatorname{vec} \mathbf{A} - \operatorname{diag} \operatorname{vec} \frac{\mathbf{A}}{(\mathbf{K} \mathbf{V}^{(\ell-1)})^2} \cdot (I_S \otimes \mathbf{K}) \cdot d \operatorname{vec} \mathbf{V}^{(\ell-1)}, \\
 d \operatorname{vec} \mathbf{V}^{(\ell)} &= \operatorname{diag} \operatorname{vec} \frac{1}{\mathbf{K}^\top \mathbf{U}^{(\ell)}} \cdot (\mathbf{1}_S \otimes I_N) \cdot d \operatorname{vec} \mathbf{b}^{(\ell)} - \operatorname{diag} \operatorname{vec} \frac{\mathbf{b}^{(\ell)} \cdot \mathbf{1}_S^\top}{(\mathbf{K}^\top \mathbf{U}^{(\ell)})^2} \cdot (I_S \otimes \mathbf{K}^\top) \cdot d \operatorname{vec} \mathbf{U}^{(\ell)}.
 \end{aligned} \tag{42}$$

The only thing that is really new, compared to Section 4.2, is the differential of $\mathbf{b}^{(\ell)}$, and we leverage the fact that $\mathbf{b}^{(\ell)} = \exp \log \mathbf{b}^{(\ell)}$, then

$$\begin{aligned}
 d \operatorname{vec} \mathbf{b}^{(\ell)} &= d \operatorname{vec} \exp \log \mathbf{b}^{(\ell)} \\
 &= \operatorname{diag} \mathbf{b}^{(\ell)} \cdot d \log \mathbf{b}^{(\ell)} \\
 &= \operatorname{diag} \mathbf{b}^{(\ell)} \cdot d [(\log \mathbf{K}^\top \mathbf{U}^{(\ell)}) \cdot \mathbf{w}],
 \end{aligned}$$

since

$$d[(\log \mathbf{K}^\top \mathbf{U}^{(\ell)}) \cdot \mathbf{w}] = [d \log (\mathbf{K}^\top \mathbf{U}^{(\ell)})] \cdot \mathbf{w} + [\log (\mathbf{K}^\top \mathbf{U}^{(\ell)})] \cdot d\mathbf{w},$$

then

$$\begin{aligned} d \operatorname{vec} \log \mathbf{b}^{(\ell)} &= d \operatorname{vec} [(\log \mathbf{K}^\top \mathbf{U}^{(\ell)}) \cdot \mathbf{w}] \\ &= \operatorname{vec} [d \log (\mathbf{K}^\top \mathbf{U}^{(\ell)}) \cdot \mathbf{w}] + \operatorname{vec} [\log (\mathbf{K}^\top \mathbf{U}^{(\ell)}) \cdot d\mathbf{w}] \\ &= (\mathbf{w}^\top \otimes I_N) d \log \operatorname{vec} (\mathbf{K}^\top \mathbf{U}^{(\ell)}) + [I_1 \otimes \log (\mathbf{K}^\top \mathbf{U}^{(\ell)})] \cdot d\mathbf{w} \\ &= (\mathbf{w}^\top \otimes I_N) \cdot \operatorname{diag} \frac{1}{\operatorname{vec} (\mathbf{K}^\top \mathbf{U}^{(\ell)})} \cdot (I_S \otimes \mathbf{K}^\top) \cdot d \operatorname{vec} \mathbf{U}^{(\ell)} + \log (\mathbf{K}^\top \mathbf{U}^{(\ell)}) \cdot d\mathbf{w} \end{aligned} \quad (43)$$

For the Wasserstein Barycenter problem, we assume the weight vector $\mathbf{w}$ is fixed or given, as supposed to the Wasserstein Dictionary Learning problem where it is considered a variable to be learned, therefore its derivative is zero. However, as we will eventually derive the Jacobians w.r.t. $\mathbf{w}$ in Section 7, I will provide the updating equations for the Jacobians w.r.t. both $\mathbf{A}$ and $\mathbf{w}$.

Update 6.1 (Jacobian Updating Equations for Parallel Barycenter Algorithm w.r.t. $\mathbf{A}$).

$$D_{\mathbf{A}} \mathbf{U}^{(\ell)} = \operatorname{diag} \operatorname{vec} \frac{1}{\mathbf{K} \mathbf{V}^{(\ell-1)}} - \operatorname{diag} \operatorname{vec} \frac{\mathbf{A}}{(\mathbf{K} \mathbf{V}^{(\ell-1)})^2} \cdot (I_S \otimes \mathbf{K}) \cdot D_{\mathbf{A}} \mathbf{V}^{(\ell-1)}, \quad (44)$$

$$D_{\mathbf{A}} \mathbf{b}^{(\ell)} = \operatorname{diag} \mathbf{b}^{(\ell)} \cdot (\mathbf{w}^\top \otimes I_N) \cdot \operatorname{diag} \operatorname{vec} \frac{1}{\mathbf{K}^\top \mathbf{U}^{(\ell)}} \cdot (I_S \otimes \mathbf{K}^\top) \cdot D_{\mathbf{A}} \mathbf{U}^{(\ell)}, \quad (45)$$

$$\begin{aligned} D_{\mathbf{A}} \mathbf{V}^{(\ell)} &= \operatorname{diag} \operatorname{vec} \frac{1}{\mathbf{K}^\top \mathbf{U}^{(\ell)}} \cdot (\mathbb{1}_S \otimes I_N) \cdot D_{\mathbf{A}} \mathbf{b}^{(\ell)} \\ &\quad - \operatorname{diag} \operatorname{vec} \frac{\mathbf{b}^{(\ell)} \cdot \mathbb{1}_S^\top}{(\mathbf{K}^\top \mathbf{U}^{(\ell)})^2} \cdot (I_S \otimes \mathbf{K}^\top) \cdot D_{\mathbf{A}} \mathbf{U}^{(\ell)}. \end{aligned} \quad (46)$$

Update 6.2 (Jacobian Updating Equations for Parallel Barycenter Algorithm w.r.t. $\mathbf{w}$).

$$D_{\mathbf{w}} \mathbf{U}^{(\ell)} = - \operatorname{diag} \operatorname{vec} \frac{\mathbf{A}}{(\mathbf{K} \mathbf{V}^{(\ell-1)})^2} \cdot (I_S \otimes \mathbf{K}) \cdot D_{\mathbf{w}} \mathbf{V}^{(\ell-1)}, \quad (47)$$

$$\begin{aligned} D_{\mathbf{w}} \mathbf{b}^{(\ell)} &= \operatorname{diag} \mathbf{b}^{(\ell)} \cdot (\mathbf{w}^\top \otimes I_N) \cdot \operatorname{diag} \operatorname{vec} \frac{1}{\mathbf{K}^\top \mathbf{U}^{(\ell)}} \\ &\quad \cdot (I_S \otimes \mathbf{K}^\top) \cdot D_{\mathbf{w}} \mathbf{U}^{(\ell)} + \operatorname{diag} \mathbf{b}^{(\ell)} \cdot \log (\mathbf{K}^\top \mathbf{U}^{(\ell)}), \end{aligned} \quad (48)$$

$$\begin{aligned} D_{\mathbf{w}} \mathbf{V}^{(\ell)} &= \operatorname{diag} \operatorname{vec} \frac{1}{\mathbf{K}^\top \mathbf{U}^{(\ell)}} \cdot (\mathbb{1}_S \otimes I_N) \cdot D_{\mathbf{w}} \mathbf{b}^{(\ell)} \\ &\quad - \operatorname{diag} \operatorname{vec} \frac{\mathbf{b}^{(\ell)} \cdot \mathbb{1}_S^\top}{(\mathbf{K}^\top \mathbf{U}^{(\ell)})^2} \cdot (I_S \otimes \mathbf{K}^\top) \cdot D_{\mathbf{w}} \mathbf{U}^{(\ell)}. \end{aligned} \quad (49)$$

Algorithm 6.1 Parallel Wasserstein Barycenter Algorithm with Jacobians**Input:** $\mathbf{A} \in \Sigma_{M \times S}$, $\mathbf{C} \in \mathbb{R}^{M \times N}$, $\varepsilon > 0$.**Initialize:** $\mathbf{U} = \mathbf{1}_{M \times S}$, $\mathbf{V}_{N \times S} = \mathbf{1}_{N \times S}$, $\mathbf{b} = \mathbf{0}_N$.**Initialize:** $\mathbf{J}_{\mathbf{U}\mathbf{A}} = \mathbf{0}_{MS \times MS}$, $\mathbf{J}_{\mathbf{V}\mathbf{A}} = \mathbf{0}_{NS \times MS}$, $\mathbf{J}_{\mathbf{b}\mathbf{A}} = \mathbf{0}_{N \times MS}$.**Initialize:** $\mathbf{J}_{\mathbf{U}\mathbf{w}} = \mathbf{0}_{MS \times S}$, $\mathbf{J}_{\mathbf{V}\mathbf{w}} = \mathbf{0}_{NS \times S}$, $\mathbf{J}_{\mathbf{b}\mathbf{w}} = \mathbf{0}_{N \times S}$.

```

1:  $\mathbf{K} = \exp(-\frac{\mathbf{C}}{\varepsilon})$ 
2: while Not Convergence do
3:   # update U and Jacobian of U
4:    $\mathbf{U} = \mathbf{A} \oslash (\mathbf{K}\mathbf{V})$ 
5:    $\mathbf{J}_{\mathbf{U}\mathbf{A}}$  from Equation (44),  $\mathbf{J}_{\mathbf{U}\mathbf{w}}$  from Equation (47)
6:
7:   # update b and Jacobian of b
8:    $\mathbf{b} = \Pi_{row}(\mathbf{K}^\top \mathbf{U})^{\mathbf{1}_N \cdot \mathbf{w}^\top}$ 
9:    $\mathbf{J}_{\mathbf{b}\mathbf{A}}$  from Equation (45),  $\mathbf{J}_{\mathbf{b}\mathbf{w}}$  from Equation (48)
10:
11:  # update V and Jacobian of V
12:   $\mathbf{V} = \mathbf{b} \oslash (\mathbf{K}^\top \mathbf{U})$ 
13:   $\mathbf{J}_{\mathbf{V}\mathbf{A}}$  from Equation (46),  $\mathbf{J}_{\mathbf{V}\mathbf{w}}$  from Equation (49)
14: end while

```

Output: $\mathbf{b}$, $\mathbf{J}_{\mathbf{b}\mathbf{A}}$, $\mathbf{J}_{\mathbf{b}\mathbf{w}}$ **6.2 Jacobians for Log-Stabilized Algorithm**

Given the updating equations in Equation (38) we get

$$\begin{aligned}
d \operatorname{vec} \mathbf{F}^{(\ell)} &= d \operatorname{vec} \mathbf{F}^{(\ell-1)} + \varepsilon \log \operatorname{diag} \frac{1}{\operatorname{vec} \mathbf{A}} d \operatorname{vec} \mathbf{A} + d \operatorname{vec} \operatorname{Min}_\varepsilon^{row} \mathbf{R}(\mathbf{F}^{(\ell-1)}, \mathbf{G}^{(\ell-1)}), \\
d \operatorname{vec} \mathbf{G}^{(\ell)} &= d \operatorname{vec} \mathbf{G}^{(\ell-1)} + \varepsilon (\mathbf{1}_S^\top \otimes I_N) \cdot d \log \mathbf{b}^{(\ell)} + d \operatorname{vec} \operatorname{Min}_\varepsilon^{col} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}),
\end{aligned}$$

and from Equation (41) we have

$$\begin{aligned}
d \log \mathbf{b}^{(\ell)} &= -\frac{1}{\varepsilon} \left[d \mathbf{G}^{(\ell-1)} \cdot \mathbf{w} + \mathbf{G}^{(\ell-1)} \cdot d \mathbf{w} + d \operatorname{Min}_\varepsilon^{col} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}) \cdot \mathbf{w} \right. \\
&\quad \left. + \operatorname{Min}_\varepsilon^{col} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}) \cdot d \mathbf{w} \right],
\end{aligned}$$

and since

$$\begin{aligned}
\operatorname{vec}(d \mathbf{G}^{(\ell-1)} \cdot \mathbf{w}) &= (\mathbf{w}^\top \otimes I_N) \cdot d \operatorname{vec} \mathbf{G}^{(\ell-1)}, \\
\operatorname{vec}(\mathbf{G}^{(\ell-1)} \cdot d \mathbf{w}) &= \mathbf{G}^{(\ell-1)} \cdot d \mathbf{w}, \\
\operatorname{vec}(d \operatorname{Min}_\varepsilon^{col} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}) \cdot \mathbf{w}) &= (\mathbf{w}^\top \otimes I_N) \cdot d \operatorname{vec} \operatorname{Min}_\varepsilon^{col} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}), \\
\operatorname{vec}(\operatorname{Min}_\varepsilon^{col} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}) \cdot d \mathbf{w}) &= \operatorname{Min}_\varepsilon^{col} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}) \cdot d \mathbf{w},
\end{aligned}$$

we have the differential of $\log \mathbf{b}^{(\ell)}$

$$\begin{aligned} d \operatorname{vec} \log \mathbf{b}^{(\ell)} &= -\frac{1}{\varepsilon} (\mathbf{w}^\top \otimes I_N) \cdot [d \operatorname{vec} \mathbf{G}^{(\ell-1)} + d \operatorname{vec} \operatorname{Min}_\varepsilon^{\operatorname{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)})] \\ &\quad - \frac{1}{\varepsilon} [\mathbf{G}^{(\ell-1)} + \operatorname{Min}_\varepsilon^{\operatorname{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)})] \cdot d \mathbf{w}. \end{aligned}$$

Update 6.3 (Jacobian Updating Equations for Log-Stabilized Barycenter Algorithm w.r.t. $\mathbf{A}$).

$$D_{\mathbf{A}} \mathbf{F}^{(\ell)} = D_{\mathbf{A}} \mathbf{F}^{(\ell-1)} + \varepsilon \operatorname{diag} \frac{1}{\operatorname{vec} \mathbf{A}} + D_{\mathbf{A}} \operatorname{Min}_\varepsilon^{\operatorname{row}} \mathbf{R}(\mathbf{F}^{(\ell-1)}, \mathbf{G}^{(\ell-1)}), \quad (50)$$

$$D_{\mathbf{A}} \log \mathbf{b}^{(\ell)} = -\frac{1}{\varepsilon} (\mathbf{w}^\top \otimes I_N) \cdot [D_{\mathbf{A}} \mathbf{G}^{(\ell-1)} + D_{\mathbf{A}} \operatorname{Min}_\varepsilon^{\operatorname{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)})], \quad (51)$$

$$D_{\mathbf{A}} \mathbf{G}^{(\ell)} = D_{\mathbf{A}} \mathbf{G}^{(\ell-1)} + \varepsilon (\mathbb{1}_S \otimes I_N) \cdot D_{\mathbf{A}} \log \mathbf{b}^{(\ell)} + D_{\mathbf{A}} \operatorname{Min}_\varepsilon^{\operatorname{row}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}). \quad (52)$$

Update 6.4 (Jacobian Updating Equations for Log-Stabilized Barycenter Algorithm w.r.t. $\mathbf{w}$).

$$D_{\mathbf{w}} \mathbf{F}^{(\ell)} = D_{\mathbf{w}} \mathbf{F}^{(\ell-1)} + D_{\mathbf{w}} \operatorname{Min}_\varepsilon^{\operatorname{row}} \mathbf{R}(\mathbf{F}^{(\ell-1)}, \mathbf{G}^{(\ell-1)}), \quad (53)$$

$$\begin{aligned} D_{\mathbf{w}} \log \mathbf{b}^{(\ell)} &= -\frac{1}{\varepsilon} (\mathbf{w}^\top \otimes I_N) \cdot [D_{\mathbf{w}} \mathbf{G}^{(\ell-1)} + D_{\mathbf{w}} \operatorname{Min}_\varepsilon^{\operatorname{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)})] \\ &\quad - \frac{1}{\varepsilon} [\mathbf{G}^{(\ell-1)} + \operatorname{Min}_\varepsilon^{\operatorname{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)})], \end{aligned} \quad (54)$$

$$D_{\mathbf{w}} \mathbf{G}^{(\ell)} = D_{\mathbf{w}} \mathbf{G}^{(\ell-1)} + \varepsilon (\mathbb{1}_S \otimes I_N) \cdot D_{\mathbf{w}} \log \mathbf{b}^{(\ell)} + D_{\mathbf{w}} \operatorname{Min}_\varepsilon^{\operatorname{row}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}). \quad (55)$$

All that remains to show is the Jacobians of the *soft-min* functions. To see this, notice that

$$\begin{aligned} D_{\mathbf{A}} \operatorname{Min}_\varepsilon^{\operatorname{row}} \mathbf{R}(\mathbf{F}^{(\ell-1)}, \mathbf{G}^{(\ell-1)}) &= \begin{bmatrix} D_{\mathbf{A}} \operatorname{Min}_\varepsilon^{\operatorname{row}} \mathbf{R}(\mathbf{f}_1^{(\ell-1)}, \mathbf{g}_1^{(\ell-1)}) \\ \vdots \\ D_{\mathbf{A}} \operatorname{Min}_\varepsilon^{\operatorname{row}} \mathbf{R}(\mathbf{f}_s^{(\ell-1)}, \mathbf{g}_s^{(\ell-1)}) \\ \vdots \\ D_{\mathbf{A}} \operatorname{Min}_\varepsilon^{\operatorname{row}} \mathbf{R}(\mathbf{f}_S^{(\ell-1)}, \mathbf{g}_S^{(\ell-1)}) \end{bmatrix} \\ &= \begin{bmatrix} D_{\mathbf{R}} \operatorname{Min}_\varepsilon^{\operatorname{row}} \mathbf{R}(\mathbf{f}_1^{(\ell-1)}, \mathbf{g}_1^{(\ell-1)}) \cdot D_{\mathbf{A}} \mathbf{R}(\mathbf{f}_1^{(\ell-1)}, \mathbf{g}_1^{(\ell-1)}) \\ \vdots \\ D_{\mathbf{R}} \operatorname{Min}_\varepsilon^{\operatorname{row}} \mathbf{R}(\mathbf{f}_s^{(\ell-1)}, \mathbf{g}_s^{(\ell-1)}) \cdot D_{\mathbf{A}} \mathbf{R}(\mathbf{f}_s^{(\ell-1)}, \mathbf{g}_s^{(\ell-1)}) \\ \vdots \\ D_{\mathbf{R}} \operatorname{Min}_\varepsilon^{\operatorname{row}} \mathbf{R}(\mathbf{f}_S^{(\ell-1)}, \mathbf{g}_S^{(\ell-1)}) \cdot D_{\mathbf{A}} \mathbf{R}(\mathbf{f}_S^{(\ell-1)}, \mathbf{g}_S^{(\ell-1)}) \end{bmatrix}, \end{aligned} \quad (56)$$

where

$$D_{\mathbf{A}} \mathbf{R}(\mathbf{f}_s, \mathbf{g}_s) = -(\mathbb{1}_N \otimes I_M) \cdot D_{\mathbf{A}} \mathbf{f}_s - (I_N \otimes \mathbb{1}_M) \cdot D_{\mathbf{A}} \mathbf{g}_s,$$

with $D_{\mathbf{A}} \mathbf{f}_s \in \mathbb{R}^{M \times MS}$ and $D_{\mathbf{A}} \mathbf{g}_s \in \mathbb{R}^{N \times MS}$ given by

$$D_{\mathbf{A}} \mathbf{F} = \begin{bmatrix} D_{\mathbf{A}} \mathbf{f}_1 \\ \vdots \\ D_{\mathbf{A}} \mathbf{f}_s \\ \vdots \\ D_{\mathbf{A}} \mathbf{f}_S \end{bmatrix}, D_{\mathbf{A}} \mathbf{G} = \begin{bmatrix} D_{\mathbf{A}} \mathbf{g}_1 \\ \vdots \\ D_{\mathbf{A}} \mathbf{g}_s \\ \vdots \\ D_{\mathbf{A}} \mathbf{g}_S \end{bmatrix}.$$

Similarly, we have that

$$D_{\mathbf{A}} \text{Min}_{\varepsilon}^{\text{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}) = \begin{bmatrix} D_{\mathbf{R}} \text{Min}_{\varepsilon}^{\text{col}} \mathbf{R}(\mathbf{f}_1^{(\ell)}, \mathbf{g}_1^{(\ell-1)}) \cdot D_{\mathbf{A}} \mathbf{R}(\mathbf{f}_1^{(\ell)}, \mathbf{g}_1^{(\ell-1)}) \\ \vdots \\ D_{\mathbf{R}} \text{Min}_{\varepsilon}^{\text{col}} \mathbf{R}(\mathbf{f}_s^{(\ell)}, \mathbf{g}_s^{(\ell-1)}) \cdot D_{\mathbf{A}} \mathbf{R}(\mathbf{f}_s^{(\ell)}, \mathbf{g}_s^{(\ell-1)}) \\ \vdots \\ D_{\mathbf{R}} \text{Min}_{\varepsilon}^{\text{col}} \mathbf{R}(\mathbf{f}_S^{(\ell)}, \mathbf{g}_S^{(\ell-1)}) \cdot D_{\mathbf{A}} \mathbf{R}(\mathbf{f}_S^{(\ell)}, \mathbf{g}_S^{(\ell-1)}) \end{bmatrix}, \quad (57)$$

$$D_{\mathbf{W}} \text{Min}_{\varepsilon}^{\text{row}} \mathbf{R}(\mathbf{F}^{(\ell-1)}, \mathbf{G}^{(\ell-1)}) = \begin{bmatrix} D_{\mathbf{R}} \text{Min}_{\varepsilon}^{\text{row}} \mathbf{R}(\mathbf{f}_1^{(\ell-1)}, \mathbf{g}_1^{(\ell-1)}) \cdot D_{\mathbf{W}} \mathbf{R}(\mathbf{f}_1^{(\ell-1)}, \mathbf{g}_1^{(\ell-1)}) \\ \vdots \\ D_{\mathbf{R}} \text{Min}_{\varepsilon}^{\text{row}} \mathbf{R}(\mathbf{f}_s^{(\ell-1)}, \mathbf{g}_s^{(\ell-1)}) \cdot D_{\mathbf{W}} \mathbf{R}(\mathbf{f}_s^{(\ell-1)}, \mathbf{g}_s^{(\ell-1)}) \\ \vdots \\ D_{\mathbf{R}} \text{Min}_{\varepsilon}^{\text{row}} \mathbf{R}(\mathbf{f}_S^{(\ell-1)}, \mathbf{g}_S^{(\ell-1)}) \cdot D_{\mathbf{W}} \mathbf{R}(\mathbf{f}_S^{(\ell-1)}, \mathbf{g}_S^{(\ell-1)}) \end{bmatrix} \quad (58)$$

$$D_{\mathbf{W}} \text{Min}_{\varepsilon}^{\text{col}} \mathbf{R}(\mathbf{F}^{(\ell)}, \mathbf{G}^{(\ell-1)}) = \begin{bmatrix} D_{\mathbf{R}} \text{Min}_{\varepsilon}^{\text{col}} \mathbf{R}(\mathbf{f}_1^{(\ell)}, \mathbf{g}_1^{(\ell-1)}) \cdot D_{\mathbf{W}} \mathbf{R}(\mathbf{f}_1^{(\ell)}, \mathbf{g}_1^{(\ell-1)}) \\ \vdots \\ D_{\mathbf{R}} \text{Min}_{\varepsilon}^{\text{col}} \mathbf{R}(\mathbf{f}_s^{(\ell)}, \mathbf{g}_s^{(\ell-1)}) \cdot D_{\mathbf{W}} \mathbf{R}(\mathbf{f}_s^{(\ell)}, \mathbf{g}_s^{(\ell-1)}) \\ \vdots \\ D_{\mathbf{R}} \text{Min}_{\varepsilon}^{\text{col}} \mathbf{R}(\mathbf{f}_S^{(\ell)}, \mathbf{g}_S^{(\ell-1)}) \cdot D_{\mathbf{W}} \mathbf{R}(\mathbf{f}_S^{(\ell)}, \mathbf{g}_S^{(\ell-1)}) \end{bmatrix}. \quad (59)$$

Algorithm 6.2 Log-Stabilized Wasserstein Barycenter Algorithm with Jacobians

Input: $\mathbf{A} \in \Sigma_{M \times S}$, $\mathbf{C} \in \mathbb{R}^{M \times N}$, $\mathbf{w} \in \Sigma_S$, $\varepsilon > 0$.
Initialize: $\mathbf{F} = \mathbf{0}_{M \times S}$, $\mathbf{G}_{N \times S} = \mathbf{1}_{N \times S}$, $\mathbf{logb} = \mathbf{0}_N$.
Initialize: $\mathbf{J}_{\mathbf{FA}} = \mathbf{0}_{MS \times MS}$, $\mathbf{J}_{\mathbf{GA}} = \mathbf{0}_{NS \times MS}$, $\mathbf{J}_{\mathbf{logbA}} = \mathbf{0}_{N \times MS}$.
Initialize: $\mathbf{J}_{\mathbf{Fw}} = \mathbf{0}_{MS \times S}$, $\mathbf{J}_{\mathbf{Gw}} = \mathbf{0}_{NS \times S}$, $\mathbf{J}_{\mathbf{logbw}} = \mathbf{0}_{N \times S}$.

- 1: **while** Not Convergence **do**
- 2: # update F and Jacobian of F
- 3: $\mathbf{R}_{min}^{row}$ from Equation (39)
- 4: $\mathbf{F} = \mathbf{F} + \varepsilon \log \mathbf{A} + \mathbf{R}_{min}^{row}$
- 5: $\mathbf{J}_{\mathbf{RowA}}$ from Equation (56), $\mathbf{J}_{\mathbf{Roww}}$ from Equation (58)
- 6: $\mathbf{J}_{\mathbf{FA}} = \mathbf{J}_{\mathbf{FA}} + \varepsilon \text{diag} \frac{1}{\text{vec } \mathbf{A}} + \mathbf{J}_{\mathbf{RowA}}$, $\mathbf{J}_{\mathbf{Fw}} = \mathbf{J}_{\mathbf{Fw}} + \mathbf{J}_{\mathbf{Roww}}$
- 7:
- 8: # update logb and Jacobian of logb
- 9: $\mathbf{R}_{min}^{col}$ from Equation (40)
- 10: $\mathbf{logb} = -\frac{1}{\varepsilon} \mathbf{G} \cdot \mathbf{w} - \frac{1}{\varepsilon} \mathbf{R}_{min}^{col} \cdot \mathbf{w}$
- 11: $\mathbf{J}_{\mathbf{RcolA}}$ from Equation (57), $\mathbf{J}_{\mathbf{Rcolw}}$ from Equation (59)
- 12: $\mathbf{J}_{\mathbf{logbA}} = -\frac{1}{\varepsilon} (\mathbf{w}^\top \otimes I_N) \cdot (\mathbf{J}_{\mathbf{GA}} + \mathbf{J}_{\mathbf{RcolA}})$
- 13: $\mathbf{J}_{\mathbf{logbw}} = -\frac{1}{\varepsilon} (\mathbf{w}^\top \otimes I_N) \cdot (\mathbf{J}_{\mathbf{Gw}} + \mathbf{J}_{\mathbf{Rcolw}}) - \frac{1}{\varepsilon} (\mathbf{G} + \mathbf{R}_{min}^{col})$
- 14:
- 15: # update G and Jacobian of G
- 16: $\mathbf{G} = \mathbf{G} + \varepsilon \mathbf{logb} \cdot \mathbf{1}_S^\top + \mathbf{R}_{min}^{col}$
- 17: $\mathbf{J}_{\mathbf{GA}} = \mathbf{J}_{\mathbf{GA}} + \varepsilon (\mathbf{1}_S \otimes I_N) \cdot \mathbf{J}_{\mathbf{logbA}} + \mathbf{J}_{\mathbf{RcolA}}$, $\mathbf{J}_{\mathbf{Gw}} = \mathbf{J}_{\mathbf{Gw}} + \varepsilon (\mathbf{1}_S \otimes I_N) \cdot \mathbf{J}_{\mathbf{logbw}} + \mathbf{J}_{\mathbf{Rcolw}}$
- 18: **end while**
- 19: $\mathbf{b} = \exp \mathbf{logb}$
- 20: $\mathbf{J}_{\mathbf{bA}} = \text{diag } \mathbf{b} \cdot \mathbf{J}_{\mathbf{logbA}}$
- 21: $\mathbf{J}_{\mathbf{bw}} = \text{diag } \mathbf{b} \cdot \mathbf{J}_{\mathbf{logbw}}$

Output: $\mathbf{b}$, $\mathbf{J}_{\mathbf{bA}}$, $\mathbf{J}_{\mathbf{bw}}$.

7 Wasserstein Dictionary Learning

7.1 Numeric Optimization Methods

Before moving on to discuss the Wasserstein Dictionary Learning algorithm, let us briefly review some gradient-based optimization algorithms commonly employed in machine learning applications.

The most famous algorithm for solving an unconstrained optimization problem is probably gradient descent method. This celebrated method can be attributed to Cauchy (1847), even though he didn't believe it will actually find the minimum nor did he show its convergence (Lemaréchal, 2012). Hadamard (1908) also seemed to have proposed this idea independently, following the work of Hilbert (1900), in the context of solving differential equations, as an alternative to Rayleigh-Ritz method (Rayleigh, 1896; Ritz, 1909; Courant, 1943).

Since gradient descent method only leverages gradient information, it is considered as first-order optimization method. Though there exist other optimization methods in numeric optimization, such as second-order methods¹⁴ or global optimization methods¹⁵ among others, it is the stochastic gradient descent (SGD) method that is *de facto* the only method¹⁶ being used in modern machine learning applications. Its history can be traced back to the stochastic approximation method (Robbins and Monro, 1951), its update (Kiefer and Wolfowitz, 1952), and was used in the perceptron model (Rosenblatt, 1958).

What is now commonly employed is the mini-batch SGD, where a batch of sample are evaluated and their gradients averaged as the current iteration’s estimate for the gradient of the objective function. Then this average gradient is used in some first-order optimization algorithm updating rule to update the parameters for the model. The procedure repeats until convergence.

Here, I list the parameter updating rules for several commonly employed optimizers¹⁷, which are implemented in the *wig* package.

Definition 5 (Dekel, Gilad-Bachrach, Shamir, and Xiao, 2012, Mini-Batch Gradient). *For some batch size B and objective function to be minimized $f(\mathbf{x}; \boldsymbol{\theta}_t)$ with data $\mathbf{x}$ and parameters $\boldsymbol{\theta}_t$ at timestamp t , define the mini-batch gradient as*

$$\mathbf{g}_t = \frac{1}{B} \sum_{b=1}^B \nabla_{\boldsymbol{\theta}} f(\mathbf{x}_b; \boldsymbol{\theta}_t), \quad (60)$$

where $\{\mathbf{x}_b\}_{b=1}^B$ denote the sequence of data.

Update 7.1 (Stochastic Gradient Descent). *Given a function to be minimized $f(\mathbf{x}; \boldsymbol{\theta}_t)$ and a mini-batched gradient $\mathbf{g}_{t-1}$ with current parameters $\boldsymbol{\theta}_{t-1}$ and learning rate η ,*

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \cdot \mathbf{g}_{t-1}.$$

Update 7.2 (Kingma and Ba, 2015, Adam). *Given a function to be minimized $f(\mathbf{x}; \boldsymbol{\theta}_t)$ and*

¹⁴For example, Newton’s method (Nocedal and Wright, 2006).

¹⁵E.g. Differential Evolution algorithm (Storn and Price, 1997) and Simulated Annealing (Kirkpatrick, Gelatt, and Vecchi, 1983).

¹⁶One of the most popular neural network library, PyTorch (Paszke et al., 2017), only includes first-order optimizers, and they are all variants of SGD with the exception of the celebrated L-BFGS method (Liu and Nocedal, 1989), which is a variant of the seminal BFGS method (Broyden, 1970; Fletcher, 1970; Goldfarb, 1970; Shanno, 1970). Among all the SGD variants, Adam optimizer (Kingma and Ba, 2015) is the most common one, simply because of its empirical performance; in fact, all other SGD-based optimizers are either derived from Adam, or its predecessors. The list of optimizers implemented in PyTorch (<https://pytorch.org/docs/stable/optim.html#algorithms>, version 2.4 as of this writing) are accessed on August 10, 2024.

¹⁷Interested readers should refer to Nocedal and Wright (2006) for a general treatment on the field of numeric optimization.

a mini-batched gradient $\mathbf{g}_{t-1}$ with current parameters $\boldsymbol{\theta}_{t-1}$ and learning rate η ,

$$\begin{aligned}\mathbf{m}_t &= \beta_1 \cdot \mathbf{m}_{t-1} + (1 - \beta_1) \cdot \mathbf{g}_{t-1}, \\ \mathbf{v}_t &= \beta_2 \cdot \mathbf{v}_{t-1} + (1 - \beta_2) \cdot \mathbf{g}_{t-1}^2, \\ \hat{\mathbf{m}}_t &= \frac{\mathbf{m}_t}{1 - \beta_1^t}, \\ \hat{\mathbf{v}}_t &= \frac{\mathbf{v}_t}{1 - \beta_2^t}, \\ \boldsymbol{\theta}_t &= \boldsymbol{\theta}_{t-1} - \eta \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon},\end{aligned}$$

where $(\cdot)^2$ denote element-wise power of 2.

Update 7.3 (Loshchilov and Hutter, 2019, AdamW). Given a function to be minimized $f(\mathbf{x}; \boldsymbol{\theta}_t)$ and a mini-batched gradient $\mathbf{g}_{t-1}$ with current parameters $\boldsymbol{\theta}_{t-1}$ decay parameter τ , and learning rate η ,

$$\begin{aligned}\hat{\mathbf{g}}_t &= \mathbf{g}_{t-1} + \tau \cdot \boldsymbol{\theta}_{t-1}, \\ \mathbf{m}_t &= \beta_1 \cdot \mathbf{m}_{t-1} + (1 - \beta_1) \cdot \hat{\mathbf{g}}_t, \\ \mathbf{v}_t &= \beta_2 \cdot \mathbf{v}_{t-1} + (1 - \beta_2) \cdot \hat{\mathbf{g}}_t^2, \\ \hat{\mathbf{m}}_t &= \frac{\mathbf{m}_t}{1 - \beta_1^t}, \\ \hat{\mathbf{v}}_t &= \frac{\mathbf{v}_t}{1 - \beta_2^t}, \\ \boldsymbol{\theta}_t &= \boldsymbol{\theta}_t - \eta_t \cdot \left(\eta \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \tau \cdot \boldsymbol{\theta}_{t-1} \right),\end{aligned}$$

where η_t is some schedule multiplier that can be fixed or decreasing to change the step size dynamically.

7.2 Wasserstein Dictionary Learning

In the Wasserstein barycenter problem, we have the data as sequence of probability vectors $\{\mathbf{a}_s\}_{s=1}^S$, and would like to obtain a “centroid” of the data $\mathbf{b}$. Here, $\mathbf{a}_s$ ’s are known as data, but $\mathbf{b}$ is unknown and thus need to be estimated or computed by optimization. The so-called Wasserstein Dictionary Learning (Schmitz et al., 2018) is the almost converse problem: suppose we have the observed $\mathbf{b}$ ’s, how to find the latent and thus unknown $\mathbf{a}_s$ such that the barycenter from which would reconstruct the data $\mathbf{b}$. In this case, it is the data $\mathbf{b}$ that is known, but not $\mathbf{a}_s$ ’s.

The Wasserstein Dictionary Learning can be considered broadly as one of the (nonlinear) topic models (Blei and Lafferty, 2009), the most famous of which would probably be Latent Dirichlet Allocation model (Blei, Ng, and Jordan, 2003, LDA). As is the case with all other topic models, we have a sequence of data, and we need to discover the common “factors” or “topics” among them, either in natural language processing (Xu, Wang, Liu, and Carin,

2018) or image processing (Schmitz et al., 2018) settings. In this note, however, I will only be discussing WDL for NLP applications.

In order to apply the Wasserstein Barycenter algorithm as in Section 5, we will need the distance matrix ($\mathbf{C}$ in Section 5), but not anymore of $\mathbf{A}$ and $\mathbf{w}$ as they are considered “trainable” parameters now with the Jacobians from Section 6.

To state the problem, first suppose that we have M documents with a total of N tokens, then we can let $\mathbf{Y} \in \mathbb{R}_+^{N \times M}$ be the matrix containing the data, i.e. M probability vectors each of size N , and thus for each column $\mathbf{y}_m$ we have $\mathbf{y}_m \in \Sigma_N$ for $m = 1, \dots, M$. To obtain the only external variable, distance matrix $\mathbf{C} \in \mathbb{R}^{N \times N}$, we will need the Euclidean distance between the word embedding vectors for any pair of tokens. In other words, we need to train the Word2Vec (Mikolov, Chen, Corrado, and Dean, 2013) network and obtain the embedding vectors $\mathbf{x}_i$ for each of the token in the dataset, $i = 1, \dots, N$. Then the entries of the distance matrix $\mathbf{C}_{i,j} = d(\mathbf{x}_i, \mathbf{x}_j)$ where $i = 1, \dots, N$, $j = 1, \dots, N$, and $d(\cdot, \cdot)$ is usually considered Euclidean (Xu, Wang, Liu, and Carin, 2018; Xie, 2020b).

For the latent topic¹⁸ matrix $\mathbf{A} \in \mathbb{R}_+^{N \times S}$ where each column $\mathbf{a}_s \in \Sigma_N$, and the weight matrix $\mathbf{W} \in \mathbb{R}_+^{S \times M}$ where each column $\mathbf{w}_m \in \Sigma_S$, we have

$$\begin{aligned} & \min_{\mathbf{A}, \mathbf{W}} \sum_{m=1}^M \mathcal{L}(\hat{\mathbf{b}}_m, \mathbf{b}_m), \\ & \text{s.t. } \hat{\mathbf{b}}_m = \arg \min_{\mathbf{b} \in \Sigma_N} \sum_{s=1}^S \mathbf{W}_{sm} \ell_{\mathbf{C}}^{\epsilon}(\mathbf{a}_s, \mathbf{b}), \\ & \mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_S] \in \mathbb{R}_+^{N \times S}, \text{ where } \mathbf{a}_s \in \Sigma_N, \\ & \mathbf{W} = [\mathbf{w}_1, \dots, \mathbf{w}_M] \in \mathbb{R}_+^{S \times M}, \text{ where } \mathbf{w}_m \in \Sigma_S. \end{aligned} \tag{61}$$

Essentially, we are considering the $\hat{\mathbf{b}}_m$, the computed barycenter from the current $\mathbf{A}$, as the reconstruction for the true $\mathbf{b}_m$ that we observe in the data. Then we will need to optimize $\mathbf{A}$ (and also implicitly $\mathbf{W}$) such that the reconstructions approximate the original data well. This problem, however, is not guaranteed to be convex (Schmitz et al., 2018), whether jointly in $\mathbf{A}$ and $\mathbf{W}$ or for each separately, unlike the original Sinkhorn problem discussed in Section 2. Thus, we aim to solve this problem by a gradient descent approach to find a local minimum, as is the usual case in machine learning literature. In order to do so, we will take the gradient of the loss with respect to the parameters (here $\mathbf{A}$ and $\mathbf{W}$), and then optimize them by some learning rate, and then repeat until some stopping criterion. The gradient computation can be easily carried out by any modern Automatic Differentiation system¹⁹, but in this note, I manually derive the gradients and Jacobians of all algorithms

¹⁸Sometimes also called atom or factor.

¹⁹They are usually embedded within neural network libraries, e.g. PyTorch (Paszke et al., 2017), TensorFlow (Abadi et al., 2016), etc., but can also be found as stand-alone libraries, e.g. autodiff in C++ (Leal, 2018) or Zygote in Julia (Innes, 2019). Unlike Symbolic or Numeric Differentiation, Automatic Differentiation (AD) essentially relies on chain’s rule and could provide accurate gradient computation. There are usually two modes for AD systems: forward mode or reverse mode, and each has their own advantages. To see this,

(in Sections 4 and 6) to achieve memory-efficient and faster code.

Note, however, we cannot directly optimize the parameters from Equation (61), since the problem is a constrained optimization problem with two constraints on the $\mathbf{a}_s$ and $\mathbf{w}_m$ being in their respective simplices. Therefore, we need to convert this constrained optimization problem into an unconstrained one (Schmitz et al., 2018; Xu, Wang, Liu, and Carin, 2018): that is, to let $\mathbf{a}_s = \text{softmax}(\boldsymbol{\alpha}_s)$ and $\mathbf{w}_m = \text{softmax}(\boldsymbol{\lambda}_m)$, as the softmax function will ensure its output to be a probability vector, i.e. $\mathbf{a}_s \in \Sigma_N$ and $\mathbf{w}_m \in \Sigma_S$. Thus, we have transformed the problem as an optimization problem with parameters $\boldsymbol{\alpha}$ and $\boldsymbol{\lambda}$. Now we need to define the softmax function.

From the *softmax* functions defined in Lemmas 10 and 11 and their Jacobians, we can enforce the constraints $\mathbf{a}_s \in \Sigma_N$ and $\mathbf{w}_m \in \Sigma_S$ by the following change of variables

$$\mathbf{A} = \text{softmax}(\boldsymbol{\alpha}), \quad \text{and} \quad \mathbf{W} = \text{softmax}(\boldsymbol{\lambda}), \quad (62)$$

where $\boldsymbol{\alpha} = [\boldsymbol{\alpha}_1, \dots, \boldsymbol{\alpha}_S] \in \mathbb{R}^{N \times S}$, and $\boldsymbol{\lambda} = [\boldsymbol{\lambda}_1, \dots, \boldsymbol{\lambda}_M] \in \mathbb{R}^{S \times M}$. Let us slightly abuse the notation to denote the vector of losses across the topics

$$\ell_{\mathbf{C}}^{\varepsilon}(\mathbf{A}, \mathbf{B}) = [\ell_{\mathbf{C}}^{\varepsilon}(\mathbf{a}_1, \mathbf{b}), \dots, \ell_{\mathbf{C}}^{\varepsilon}(\mathbf{a}_S, \mathbf{b})]^{\top} \in \mathbb{R}^N.$$

Note that $\ell_{\mathbf{C}}^{\varepsilon}(\mathbf{A}, \mathbf{B})$ is essentially the same as $\ell_{\mathbf{C}}^{\varepsilon}(\mathbf{A}, \mathbf{b})$, where $\mathbf{B} = \mathbf{b} \cdot \mathbf{1}_S^{\top} \in \mathbb{R}^{N \times S}$ as the “many-to-one” transport problem in Section 3.2. Therefore, we can rewrite the problem in Equation (61) as an unconstrained one with respect to parameters $\boldsymbol{\alpha}$ and $\boldsymbol{\lambda}$

$$\min_{\boldsymbol{\alpha}, \boldsymbol{\lambda}} \sum_{m=1}^M \mathcal{L}(\hat{\mathbf{b}}_m, \mathbf{b}_m), \quad (63)$$

where $\hat{\mathbf{b}}_m$, $\mathbf{A}$, and $\mathbf{W}$ satisfy the following

$$\begin{aligned} \hat{\mathbf{b}}_m &= \arg \min_{\mathbf{b} \in \Sigma_N} \mathbf{w}_m^{\top} \cdot \ell_{\mathbf{C}}^{\varepsilon}(\mathbf{A}, \mathbf{b}), \\ \mathbf{A} &= \text{softmax}(\boldsymbol{\alpha}), \\ \mathbf{W} &= \text{softmax}(\boldsymbol{\lambda}). \end{aligned}$$

It is obvious that Equation (63) is an unconstrained optimization problem, compared to the original constrained formulation in Equation (61), since the parameters need optimizing are the auxiliary variables $\boldsymbol{\alpha}$ and $\boldsymbol{\lambda}$, and all we need to do is to obtain the gradients through the loss, and “backpropagate” to update $\boldsymbol{\alpha}$ and $\boldsymbol{\lambda}$. Thus, since we know that $\mathbf{A} = \text{softmax}(\boldsymbol{\alpha})$ and $\mathbf{W} = \text{softmax}(\boldsymbol{\lambda})$, we can retrieve the topics’ distribution $\mathbf{A}$ and the weights $\mathbf{W}$.

let a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, when $n \ll m$, then forward mode is more efficient; if, however, $n \gg m$, then reverse mode is more efficient. Neural Network libraries are usually implemented in reverse mode, since there is usually a scalar loss but with many parameters of the neural network model. Interested readers could refer to Nocedal and Wright (2006, Chapter 8) for details of AD.

7.3 WDL Algorithm

To leverage the idea of Wasserstein Barycenter and use it in a “learning” environment, i.e. Wasserstein Dictionary Learning (WDL), we will take some random initialized parameters α and λ , calculate their *softmax* column-wise to get $\mathbf{A}$ and $\mathbf{W}$, calculate their optimal barycenter, calculate the loss between the barycenter and the original data, take the gradients of the loss with respect to the parameters, and then update the parameters based on the gradients. The computation of the optimal barycenter is based on the Wasserstein barycenter problem, detailed in Section 5 with Jacobian calculation in Section 6, and the parameter-updating routine can be chosen to be any optimization technique, for example Schmitz et al. (2018) chooses L-BFGS, and Xie (2020a) uses Adam.

To lay out the algorithm of WDL, we are taking what we have built so far in this entire note. There are a few things remain to be derived. We will definitely need the Jacobian of the softmax function w.r.t. its argument, which is given in Lemma 11. Therefore, we have $D_\alpha \mathbf{A} = D_\alpha \text{softmax}(\alpha)$ and $D_\lambda \mathbf{W} = D_\lambda \text{softmax}(\lambda)$ given by Equation (66).

Next is the choice of the loss function $\mathcal{L}$. Note that this loss function is the “loss of learning problem” as in Section 7.2, as supposed to the “loss of barycenter problem” as in Sections 5 and 6. The most common choice of a loss function is obviously the squared loss function, i.e. $\mathcal{L}(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2^2$, but other losses can certainly be considered as well, as in Schmitz et al. (2018, Table 1).

Since $\nabla_{\mathbf{A}} \mathcal{L} = (D_\alpha \mathcal{L})^\top$ and $\nabla_{\mathbf{W}} \mathcal{L} = (D_\lambda \mathcal{L})^\top$, by Chain’s Rule, we have that

$$\begin{aligned} D_\alpha \mathcal{L} &= D_{\hat{\mathbf{b}}} \mathcal{L} \cdot D_{\mathbf{A}} \hat{\mathbf{b}} \cdot D_\alpha \mathbf{A}, \\ D_\lambda \mathcal{L} &= D_{\hat{\mathbf{b}}} \mathcal{L} \cdot D_{\mathbf{W}} \hat{\mathbf{b}} \cdot D_\lambda \mathbf{W}. \end{aligned}$$

Given the differential of the quadratic loss, $d(\hat{\mathbf{b}} - \mathbf{b})^2 = 2 \text{diag}(\hat{\mathbf{b}} - \mathbf{b})$, we have the Jacobian of $D_{\hat{\mathbf{b}}} \mathcal{L}$

$$D_{\hat{\mathbf{b}}} \mathcal{L} = 2 \text{diag}(\hat{\mathbf{b}} - \mathbf{b}). \quad (64)$$

The last piece is just $D_{\mathbf{A}} \hat{\mathbf{b}}$ and $D_{\mathbf{W}} \hat{\mathbf{b}}$, which are just the Jacobians of Wasserstein Barycenter algorithm as in Section 6.

Algorithm 7.1 Wasserstein Dictionary Learning Algorithm

Input: $\mathbf{Y} \in \mathbb{R}^{N \times M}$, $\mathbf{C} \in \mathbb{R}^{N \times N}$, $\varepsilon > 0$, $S \in \mathbb{N}_+$ **Initialize:** $\boldsymbol{\alpha} \in \mathbb{R}^{N \times S}$, $\boldsymbol{\lambda} \in \mathbb{R}^{S \times M}$

```

1:  $\mathbf{K} = \exp(-\frac{\mathbf{C}}{\varepsilon})$  # only used with Algorithm 6.1
2:  $\mathbf{A} = \text{softmax}(\boldsymbol{\alpha})$ ,  $\mathbf{W} = \text{softmax}(\boldsymbol{\lambda})$ ,  $\mathbf{J}_{\mathbf{A}\boldsymbol{\alpha}}$  from Equation (66) by  $\boldsymbol{\alpha}$ 
3: while Not Convergence do
4:    $\mathbf{g}_{\boldsymbol{\alpha}} = \mathbf{0}_{N \times S}$ ,  $\mathbf{g}_{\boldsymbol{\lambda}} = \mathbf{0}_S$ 
5:   for Each  $\ell = 1, \dots, B$  in Batch B do
6:     get  $\mathbf{y}^\ell \in \mathbb{R}^N$  from  $\mathbf{Y}$ , and get  $\mathbf{l} \in \mathbb{R}^S$  from  $\boldsymbol{\lambda}$ 
7:      $\mathbf{J}_{\mathbf{w}\mathbf{l}}^\ell$  from Equation (65) by  $\mathbf{l}$ 
8:      $\mathbf{b}^\ell$ ,  $\mathbf{J}_{\mathbf{b}\mathbf{A}}^\ell$ ,  $\mathbf{J}_{\mathbf{b}\mathbf{w}}^\ell$  from Algorithm 6.1 or Algorithm 6.2
9:      $\mathbf{J}_{\mathbf{l}\mathbf{b}}^\ell = 2 \text{diag}(\mathbf{b}^\ell - \mathbf{y}^\ell)$ 
10:     $\mathbf{g}_{\boldsymbol{\alpha}} = \mathbf{g}_{\boldsymbol{\alpha}} + (\mathbf{J}_{\mathbf{l}\mathbf{b}}^\ell \cdot \mathbf{J}_{\mathbf{b}\mathbf{A}}^\ell \cdot \mathbf{J}_{\mathbf{A}\boldsymbol{\alpha}})^\top$ ,  $\mathbf{g}_{\boldsymbol{\lambda}} = \mathbf{g}_{\boldsymbol{\lambda}} + (\mathbf{J}_{\mathbf{l}\mathbf{b}}^\ell \cdot \mathbf{J}_{\mathbf{b}\mathbf{w}}^\ell \cdot \mathbf{J}_{\mathbf{w}\mathbf{l}}^\ell)^\top$ 
11:   end for
12:    $\mathbf{g}_{\boldsymbol{\alpha}} = \mathbf{g}_{\boldsymbol{\alpha}}/B$ ,  $\mathbf{g}_{\boldsymbol{\lambda}} = \mathbf{g}_{\boldsymbol{\lambda}}/B$  # average gradient
13:   Update  $\boldsymbol{\alpha}$  and  $\boldsymbol{\lambda}$  by Section 7.1 with  $\mathbf{g}_{\boldsymbol{\alpha}}$  and  $\mathbf{g}_{\boldsymbol{\lambda}} \cdot \mathbf{1}_M^\top$ 
14:    $\mathbf{A} = \text{softmax}(\boldsymbol{\alpha})$ ,  $\mathbf{W} = \text{softmax}(\boldsymbol{\lambda})$ ,  $\mathbf{J}_{\mathbf{A}\boldsymbol{\alpha}}$  by Equation (66) by  $\boldsymbol{\alpha}$ 
15: end while
Output:  $\mathbf{A}$ ,  $\mathbf{W}$ 

```

References

- Abadi, Martin et al. (2016). “TensorFlow: A System for Large-Scale Machine Learning.” In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*.
- Alaya, Mokhtar Z., Maxime Bérar, Gilles Gasso, and Alain Rakotomamonjy (2019). “Screening Sinkhorn Algorithm for Regularized Optimal Transport.”
- Altschuler, Jason, Jonathan Weed, and Philippe Rigollet (2017). “Near-Linear Time Approximation Algorithms for Optimal Transport via Sinkhorn Iteration.” In: *Advances in Neural Information Processing Systems 30*. Ed. by I. Guyon et al. Curran Associates, Inc.
- Bacharach, Michael (1970). *Biproportional Matrices and Input-Output Change*. CUP Archive. 192 pp.
- Benamou, Jean-David et al. (2015). “Iterative Bregman Projections for Regularized Transportation Problems.” In: *SIAM Journal on Scientific Computing* 37.2. DOI: 10.1137/141000439.
- Blei, David M. and John D. Lafferty (2009). “Topic Models.” In: *Text Mining*. Chapman and Hall/CRC.
- Blei, David M., Andrew Y. Ng, and Michael I. Jordan (2003). “Latent Dirichlet Allocation.” In: *Journal of Machine Learning Research* 3 (Jan).
- Broyden, C. G. (1970). “The Convergence of a Class of Double-Rank Minimization Algorithms 1. General Considerations.” In: *IMA Journal of Applied Mathematics* 6.1. DOI: 10.1093/imamat/6.1.76.
- Cauchy, Augustin (1847). “Méthode Générale Pour La Résolution Des Systemes d’équations Simultanées.” In: *Comp. Rend. Sci. Paris* 25.1847.
- Courant, R. (1943). “Variational Methods for the Solution of Problems of Equilibrium and Vibrations.” In: *Bulletin of the American Mathematical Society* 49.1. DOI: 10.1090/S0002-9904-1943-078
- Cuturi, Marco (2013). “Sinkhorn Distances: Lightspeed Computation of Optimal Transport.” In: *Advances in Neural Information Processing Systems 26*. Ed. by C. J. C. Burges et al. Curran Associates, Inc.
- Dekel, Ofer, Ran Gilad-Bachrach, Ohad Shamir, and Lin Xiao (2012). “Optimal Distributed Online Prediction Using Mini-Batches.” In: *Journal of Machine Learning Research* 13.1.
- Flamary, Rémi et al. (2021). “POT: Python Optimal Transport.” In: *Journal of Machine Learning Research* 22.78.
- Fletcher, R. (1970). “A New Approach to Variable Metric Algorithms.” In: *The Computer Journal* 13.3. DOI: 10.1093/comjnl/13.3.317.
- Galichon, Alfred (2016). *Optimal Transport Methods in Economics*. Princeton University Press. 185 pp. DOI: 10.23943/princeton/9780691172767.001.0001.
- Goldfarb, Donald (1970). “A Family of Variable-Metric Methods Derived by Variational Means.” In: *Mathematics of Computation* 24.109. DOI: 10.2307/2004873.
- Hadamard, Jacques (1908). *Mémoire Sur Le Problème d’analyse Relatif à l’équilibre Des Plaques Élastiques Encastrées*. Vol. 33. Imprimerie nationale.
- Hilbert, David (1900). “Sur Le Principe de Dirichlet.” In: *Nouvelles annales de mathématiques: journal des candidats aux écoles polytechnique et normale* 19.

- Innes, Michael (2019). *Don't Unroll Adjoint: Differentiating SSA-form Programs*. DOI: 10.48550/arXiv.1811.00000. (Visited on 07/27/2022). Pre-published.
- Kantorovich, L. V. (1942). "On the Translocation of Masses." In: *Dokl. Akad. Nauk. USSR (N.S.)* 37.
- Karcher, Hermann (2014). *Riemannian Center of Mass and so Called Karcher Mean*. DOI: 10.48550/arXiv.1407.2087. (Visited on 07/28/2024). Pre-published.
- Kiefer, J. and J. Wolfowitz (1952). "Stochastic Estimation of the Maximum of a Regression Function." In: *The Annals of Mathematical Statistics* 23.3. DOI: 10.1214/aoms/1177729392.
- Kingma, Diederick P and Jimmy Ba (2015). "Adam: A Method for Stochastic Optimization." In: *International Conference on Learning Representations (ICLR)*.
- Kirkpatrick, S., C. D. Gelatt, and M. P. Vecchi (1983). "Optimization by Simulated Annealing." In: *Science* 220.4598. DOI: 10.1126/science.220.4598.671.
- Knight, P. (2008). "The Sinkhorn-Knopp Algorithm: Convergence and Applications." In: *SIAM Journal on Matrix Analysis and Applications* 30.1. DOI: 10.1137/060659624.
- Leal, Allan M. M. (2018). *Autodiff, a Modern, Fast and Expressive C++ Library for Automatic Differentiation*. <https://autodiff.github.io>.
- Lemar  chal, Claude (2012). "Cauchy and the Gradient Method." In: *Doc Math Extra* 251.254.
- L  onard, Christian (2013). "A Survey of the Schr  dinger Problem and Some of Its Connections with Optimal Transport."
- Liu, Dong C. and Jorge Nocedal (1989). "On the Limited Memory BFGS Method for Large Scale Optimization." In: *Mathematical Programming* 45.1. DOI: 10.1007/BF01589116.
- Loshchilov, Ilya and Frank Hutter (2019). *Decoupled Weight Decay Regularization*. DOI: 10.48550/arXiv.1711.05101. (Visited on 08/01/2024). Pre-published.
- Magnus, Jan R. and Heinz Neudecker (2019). *Matrix Differential Calculus with Applications in Statistics and Econometrics*. 3rd edition. Hoboken (N.J.): Wiley. 504 pp.
- Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean (2013). "Efficient Estimation of Word Representations in Vector Space." In.
- Modin, Klas (2024). *On the Geometry and Dynamical Formulation of the Sinkhorn Algorithm for Optimal Transport*. DOI: 10.48550/arXiv.2309.09089. (Visited on 08/03/2024). Pre-published.
- Monge, Gaspard (1781). "Memoire Sur La Theorie Des Deblais et Des Remblais." In: *Histoire de l'Academie Royale des Sciences de Paris*.
- Nocedal, Jorge and Stephen Wright (2006). *Numerical Optimization*. 2nd edition. New York: Springer. 686 pp.
- Paszke, Adam et al. (2017). "Automatic Differentiation in PyTorch." In: *NIPS-W*.
- Peyr  , Gabriel and Marco Cuturi (2019). "Computational Optimal Transport: With Applications to Data Science." In: *Foundations and Trends   in Machine Learning* 11.5-6. DOI: 10.1561/22000000073.
- Rayleigh, John William Strutt Baron (1896). *The Theory of Sound*. Macmillan. 526 pp.
- Revels, Jarrett, Miles Lubin, and Theodore Papamarkou (2016). *Forward-Mode Automatic Differentiation in Julia*. DOI: 10.48550/arXiv.1607.07892. (Visited on 04/11/2025). Pre-published.

- Ritz, Walter (1909). “Über eine neue Methode zur Lösung gewisser Variationsprobleme der mathematischen Physik.” In: *Journal für die reine und angewandte Mathematik* 135.
- Robbins, Herbert and Sutton Monro (1951). “A Stochastic Approximation Method.” In: *The Annals of Mathematical Statistics* 22.3. DOI: 10.1214/aoms/1177729586.
- Rosenblatt, F. (1958). “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain.” In: *Psychological Review* 65.6. DOI: 10.1037/h0042519.
- Santambrogio, Filippo (2015). *Optimal Transport for Applied Mathematicians: Calculus of Variations, PDEs, and Modeling*. Birkhäuser. 376 pp.
- Schmitz, Morgan A. et al. (2018). “Wasserstein Dictionary Learning: Optimal Transport-Based Unsupervised Nonlinear Dictionary Learning.” In: *SIAM Journal on Imaging Sciences* 11.1. DOI: 10.1137/17M1140431.
- Schrödinger, Erwin (1931). “Über Die Umkehrung Der Naturgesetze.” In: *Angewandte Chemie* 44.30. DOI: 10.1002/ange.19310443014.
- Shanno, D. F. (1970). “Conditioning of Quasi-Newton Methods for Function Minimization.” In: *Mathematics of Computation* 24.111. DOI: 10.1090/S0025-5718-1970-0274029-X.
- Sinkhorn, Richard (1964). “A Relationship Between Arbitrary Positive Matrices and Doubly Stochastic Matrices.” In: *The Annals of Mathematical Statistics* 35.2. DOI: 10.1214/aoms/1177703591.
- Sinkhorn, Richard and Paul Knopp (1967). “Concerning Nonnegative Matrices and Doubly Stochastic Matrices.” In: *Pacific Journal of Mathematics* 21.2.
- Storn, Rainer and Kenneth Price (1997). “Differential Evolution – a Simple and Efficient Heuristic for Global Optimization over Continuous Spaces.” In: *Journal of Global Optimization* 11.4. DOI: 10.1023/A:1008202821328.
- Villani, Cédric (2003). *Topics in Optimal Transportation*. Vol. 58. Graduate Studies in Mathematics. American Mathematical Society. DOI: 10.1090/gsm/058.
- Villani, Cédric (2008). *Optimal Transport: Old and New*. Springer Science & Business Media. 970 pp.
- Xie, Fangzhou (2020a). “Pruned Wasserstein Index Generation Model and Wigpy Package.” In: *CARMA 2020 - 3rd International Conference on Advanced Research Methods and Analytics*. DOI: 10.4995/CARMA2020.2020.11557.
- Xie, Fangzhou (2020b). “Wasserstein Index Generation Model: Automatic Generation of Time-Series Index with Application to Economic Policy Uncertainty.” In: *Economics Letters* 186. DOI: 10.1016/j.econlet.2019.108874.
- Xu, Hongteng, Wenlin Wang, Wei Liu, and Lawrence Carin (2018). “Distilled Wasserstein Learning for Word Embedding and Topic Modeling.” In: *Advances in Neural Information Processing Systems* 31. Ed. by S. Bengio et al. Curran Associates, Inc.

Appendices

Appendix A Mathematical Notations

Mathematical notations used in this note:

- $\mathbf{a}$: lower case letters in bold denote vectors
- $\mathbf{A}$: upper case letters in bold denote matrices
- $\mathbb{1}_n, \mathbb{0}_n$: a vector of ones (or zeros) of length n
- Σ_n : probability simplex $\Sigma_n \equiv \{\mathbf{a} \in \mathbb{R}_+^n : \sum_{i=1}^n \mathbf{a}_i = 1\}$.
- $\langle \cdot, \cdot \rangle$: inner (dot) product of two matrices of same size
- $\odot$: element-wise (Hadamard) multiplication
- $\oslash$ or $\frac{1}{\mathbf{X}}$: element-wise (Hadamard) division
- $\log(\cdot), \exp(\cdot)$: element-wise logarithm and exponential functions
- $\text{diag}(\mathbf{x})$: create a diagonal matrix $n \times n$ from vector $\mathbf{x} \in \mathbb{R}^n$
- $\text{vec } \mathbf{AB} = (I_m \otimes \mathbf{A}) \text{vec } \mathbf{B} = (\mathbf{B}^\top \otimes I_k) \text{vec } \mathbf{A}$: for $\mathbf{A} : k \times l$ and $\mathbf{B} : l \times m$
- $\mathcal{K}^{(m,n)}$: Commutation matrix such that $\mathcal{K}^{(m,n)} \text{vec } \mathbf{A} = \text{vec } (\mathbf{A}^\top)$ for a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$
- $[P]$: Iverson bracket, i.e. $[P] = 1$ if P is true, and $[P] = 0$ otherwise.
- Matrix calculus notations used throughout this note are based on Magnus and Neudecker (2019)

Appendix B Lemmas

Lemma 6. For any column vectors $\mathbf{a} \in \mathbb{R}^m$ and $\mathbf{b} \in \mathbb{R}^n$, $\mathbf{a} \otimes \mathbf{b}^\top = \mathbf{a} \cdot \mathbf{b}^\top$.

Proof of Lemma 6. To see this, we use the property of Kronecker product,

$$\mathbf{a} \otimes \mathbf{b}^\top = (\mathbf{a} \cdot I_1) \otimes (I_1 \cdot \mathbf{b}^\top) = (\mathbf{a} \otimes I_1) \cdot (I_1 \otimes \mathbf{b}^\top) = \mathbf{a} \cdot \mathbf{b}^\top.$$

□

Lemma 7. For any column vectors $\mathbf{a} \in \mathbb{R}^m$ and $\mathbf{b} \in \mathbb{R}^n$, $\mathbf{a}^\top \otimes \mathbf{b} = \mathbf{b} \cdot \mathbf{a}^\top$.

Proof of Lemma 7. To see this, we use the same property as in the previous lemma,

$$\mathbf{a}^\top \otimes \mathbf{b} = (I_1 \cdot \mathbf{a}^\top) \otimes (\mathbf{b} \cdot I_1) = (I_1 \otimes \mathbf{b}) \cdot (\mathbf{a}^\top \otimes I_1) = \mathbf{b} \cdot \mathbf{a}^\top.$$

□

Lemma 8. Given a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and two vectors $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{y} \in \mathbb{R}^n$, $\text{diag } \mathbf{x} \cdot \mathbf{A} \cdot \text{diag } \mathbf{y} = (\mathbf{x}\mathbf{y}^\top) \odot \mathbf{A}$.

Proof of Lemma 8. In the derivation of the Jacobians, we frequently encounter matrix computations of the form $\text{diag } \mathbf{x} \cdot \mathbf{A} \cdot \text{diag } \mathbf{y}$. To avoid the creation of highly sparse diagonal matrices by the $\text{diag}(\cdot)$ function from the vectors, we can compute the outer product by the two vectors first and then compute its Hadamard product with the internal matrix. Not only will this give us a formulation that is more compact, but also more efficient in the computation.

To see this, note that $\mathbf{x}\mathbf{y}^\top$ gives us an $m \times n$ matrix with each element being $(\mathbf{x}\mathbf{y}^\top)_{ij} = \mathbf{x}_i\mathbf{y}_j$, then the elements of the final Hadamard product would be $((\mathbf{x}\mathbf{y}^\top) \odot \mathbf{A})_{ij} = \mathbf{A}_{ij}\mathbf{x}_i\mathbf{y}_j$.

Now, on the other hand, if we look at the form $\text{diag } \mathbf{x} \cdot \mathbf{A} \cdot \text{diag } \mathbf{y}$, the element is also $(\text{diag } \mathbf{x} \cdot \mathbf{A} \cdot \text{diag } \mathbf{y})_{ij} = \mathbf{x}_i\mathbf{A}_{ij}\mathbf{y}_j$, since for each row i in $\mathbf{A}$, we need to multiply by $\mathbf{x}_i$ and for each column j in $\mathbf{A}$, we need to multiply by $\mathbf{y}_j$. This concludes the proof as the two forms are identical. □

Lemma 9 (Gradient of soft-minimum function). For the soft-minimum function defined in Equation (21), a vector $\mathbf{z} \in \mathbb{R}^n$ with its minimum as $\underline{\mathbf{z}}$, we have its gradient as

$$\nabla_{\mathbf{z}} \min_{\varepsilon} \mathbf{z} = \frac{1}{\mathbb{1}_n^\top \cdot \exp\left(-\frac{\mathbf{z}-\underline{\mathbf{z}}}{\varepsilon}\right)} \cdot \exp\left(-\frac{\mathbf{z}-\underline{\mathbf{z}}}{\varepsilon}\right).$$

Proof of Lemma 9. Given the soft-minimum function, we can take the differential

$$\frac{\partial \min_{\varepsilon} \mathbf{z}}{\partial \mathbf{z}_i} = \frac{\partial \underline{\mathbf{z}}}{\partial \mathbf{z}_i} - \varepsilon \frac{1}{\sum_j \exp\left(-\frac{\mathbf{z}_j - \underline{\mathbf{z}}}{\varepsilon}\right)} \cdot \frac{\partial}{\partial \mathbf{z}_i} \left[\sum_j \exp\left(-\frac{\mathbf{z}_j - \underline{\mathbf{z}}}{\varepsilon}\right) \right].$$

Then we can see that

$$\begin{aligned}
\frac{\partial}{\partial \mathbf{z}_i} \left[\sum_j \exp \left(-\frac{\mathbf{z}_j - \underline{\mathbf{z}}}{\varepsilon} \right) \right] &= \sum_j \frac{\partial}{\partial \mathbf{z}_i} \left[\exp \left(-\frac{\mathbf{z}_j - \underline{\mathbf{z}}}{\varepsilon} \right) \right] \\
&= \frac{\partial}{\partial \mathbf{z}_i} \exp \left(-\frac{\mathbf{z}_i - \underline{\mathbf{z}}}{\varepsilon} \right) + \sum_{j \neq i} \frac{\partial}{\partial \mathbf{z}_i} \left[\exp \left(-\frac{\mathbf{z}_j - \underline{\mathbf{z}}}{\varepsilon} \right) \right] \\
&= -\frac{1}{\varepsilon} \exp \left(-\frac{\mathbf{z}_i - \underline{\mathbf{z}}}{\varepsilon} \right) \left(1 - \frac{\partial \underline{\mathbf{z}}}{\partial \mathbf{z}_i} \right) + \frac{1}{\varepsilon} \sum_{j \neq i} \exp \left(-\frac{\mathbf{z}_j - \underline{\mathbf{z}}}{\varepsilon} \right) \cdot \frac{\partial \underline{\mathbf{z}}}{\partial \mathbf{z}_i} \\
&= -\frac{1}{\varepsilon} \exp \left(-\frac{\mathbf{z}_i - \underline{\mathbf{z}}}{\varepsilon} \right) + \frac{1}{\varepsilon} \sum_j \exp \left(-\frac{\mathbf{z}_j - \underline{\mathbf{z}}}{\varepsilon} \right) \cdot \frac{\partial \underline{\mathbf{z}}}{\partial \mathbf{z}_i}.
\end{aligned}$$

Putting everything together, we have

$$\frac{\partial \min_{\varepsilon} \mathbf{z}}{\partial \mathbf{z}_i} = \frac{1}{\sum_j \exp \left(-\frac{\mathbf{z}_j - \underline{\mathbf{z}}}{\varepsilon} \right)} \cdot \exp \left(-\frac{\mathbf{z}_j - \underline{\mathbf{z}}}{\varepsilon} \right).$$

One thing that worth mentioning is about $\frac{\partial \underline{\mathbf{z}}}{\partial \mathbf{z}_i}$. Even though this term was cancelled out in the final expression, it was handled differently in different AD implementations. Julia Zygote implements it with “left-most” or “first-occurrence” rule, while PyTorch implements it as “subgradient averaging” approach. $\square$

Lemma 10 (Jacobian of *softmax* function for a vector). *For the softmax function and a vector $\mathbf{x} \in \mathbb{R}^n$ with its maximum as $\bar{\mathbf{x}}$, we have*

$$\text{softmax}(\mathbf{x}) = \frac{\exp(\mathbf{x})}{\mathbb{1}_n^\top \cdot \exp(\mathbf{x})} = \frac{\exp(\mathbf{x} - \bar{\mathbf{x}})}{\mathbb{1}_n^\top \cdot \exp(\mathbf{x} - \bar{\mathbf{x}})},$$

where the second expression would be numerically stable. Moreover, we can derive its Jacobian w.r.t. $\mathbf{x}$ as

$$D_{\mathbf{x}} \text{softmax}(\mathbf{x}) = \text{diag} \text{softmax}(\mathbf{x}) - \text{softmax}(\mathbf{x}) \cdot \text{softmax}(\mathbf{x})^\top. \quad (65)$$

Proof of Lemma 10. To make the computation of the *softmax* function stable, we will need to subtract the maximum value on both the numerator and denominator and the result would be the same. Therefore, its derivative is also the same. To get the Jacobian of softmax, we have

$$\begin{aligned}
\frac{\partial \text{softmax}(\mathbf{x})_i}{\partial \mathbf{x}_j} &= \frac{\frac{\partial}{\partial \mathbf{x}_j} \exp \mathbf{x}_i \cdot \sum_j \exp \mathbf{x}_j - \exp \mathbf{x}_i \cdot \frac{\partial}{\partial \mathbf{x}_j} \sum_j \exp \mathbf{x}_j}{\left(\sum_j \exp \mathbf{x}_j \right)^2} \\
&= \frac{\exp \mathbf{x}_i}{\sum_j \exp \mathbf{x}_j} \cdot \frac{\partial \mathbf{x}_i}{\partial \mathbf{x}_j} - \frac{\exp \mathbf{x}_i}{\sum_j \exp \mathbf{x}_j} \cdot \frac{\exp \mathbf{x}_j}{\sum_j \exp \mathbf{x}_j} \\
&= \text{softmax}(\mathbf{x})_i \cdot \frac{\partial \mathbf{x}_i}{\partial \mathbf{x}_j} - \text{softmax}(\mathbf{x})_i \cdot \text{softmax}(\mathbf{x})_j.
\end{aligned}$$

Therefore, the Jacobian matrix is

$$D_{\mathbf{x}} \text{softmax}(\mathbf{x}) = \text{diag} \text{softmax}(\mathbf{x}) - \text{softmax}(\mathbf{x}) \cdot \text{softmax}(\mathbf{x})^\top.$$

□

Lemma 11 (Jacobian of *softmax* function for a matrix). *Suppose we have a matrix $\mathbf{X} \in \mathbb{R}^{m \times n}$, and let*

$$\text{softmax}(\mathbf{X}) = [\text{softmax}(\mathbf{x}_1), \dots, \text{softmax}(\mathbf{x}_n)],$$

where $\text{softmax}(\mathbf{x}_i)$ for $i = 1, \dots, n$ is from Lemma 10. Moreover, we have

$$D_{\mathbf{X}} \text{softmax}(\mathbf{X}) = \begin{bmatrix} \mathbf{J}_1 & 0 & \dots & 0 \\ 0 & \mathbf{J}_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \mathbf{J}_n \end{bmatrix} \in \mathbb{R}^{mn \times mn}, \quad (66)$$

where $\mathbf{J}_j \in \mathbb{R}^{m \times m}$ is the Jacobian of *softmax* of $\mathbf{X}_{:,j}$.

Proof of Lemma 11. Let $\mathbf{Y} = \text{softmax}(\mathbf{X}) \in \mathbb{R}^{m \times n}$, with each column $\mathbf{Y}_{:,j} = \text{softmax}(\mathbf{X}_{:,j})$, and

$$\text{softmax}(\mathbf{X}_{:,j})_i = \frac{\exp \mathbf{X}_{ij}}{\sum_{k=1}^m \exp \mathbf{X}_{kj}},$$

for $i = 1, \dots, m$ and $j = 1, \dots, n$. Since the *softmax* function is applied independently to each column, the Jacobian of $\mathbf{Y}$ w.r.t. $\mathbf{X}$ is a block-diagonal matrix of the individual *softmax* Jacobians. Therefore, the Jacobian of *softmax* $\mathbf{X}$ takes the form of Equation (66), i.e.

$$D_{\mathbf{X}} \mathbf{Y} = D_{\mathbf{X}} \text{softmax}(\mathbf{X}),$$

where $\mathbf{J}_j$ is given by Lemma 10, i.e.

$$\begin{aligned} \mathbf{J}_j &= D_{\mathbf{X}_{:,j}} \text{softmax}(\mathbf{X}_{:,j}) \\ &= \text{diag} \text{softmax}(\mathbf{X}_{:,j}) - \text{softmax}(\mathbf{X}_{:,j}) \cdot \text{softmax}(\mathbf{X}_{:,j})^\top. \end{aligned}$$

□